

SNOWFLAKE

Complete Interview Q&A; Guide

120 Questions with Detailed SQL Answers

For 5+ Years Experience

Section 1: 70 Technical Questions · Section 2: 50 Scenario-Based Questions

Architecture · Data Loading · Performance · CDC & SCD · Security · Pipelines · SQL · Cost

Table of Contents

SECTION 1 — Technical Questions (70)

- Architecture (8 questions)
- Data Loading (5 questions)
- Performance (6 questions)
- CDC & SCD (7 questions)
- Security (4 questions)
- Pipelines (5 questions)
- SQL (3 questions)
- Cost (5 questions)

SECTION 2 — Scenario-Based Questions (50)

- Performance scenarios (3)
- Data Loading scenarios (2)
- CDC & SCD scenarios (3)
- Pipeline scenarios (2)
- Security scenarios (2)
- Cost scenarios (1)
- Debugging scenarios (2)
- Architecture scenarios (3)

SECTION 1

Technical Interview Questions — 70 Q&A with SQL

These questions test deep knowledge of Snowflake internals, architecture, and best practices. Interviewers at the 5-year level expect specific technical reasoning — not just definitions. Every answer includes the exact SQL and the key insight that separates a correct answer from a great one.

ARCHITECTURE

Interview Questions — Architecture

Q1 [Easy] [Architecture]

What are the three layers of Snowflake architecture and why does separating storage from compute matter?

Answer:

Snowflake has three fully independent layers:

1. Cloud Services Layer: authentication, query optimization, metadata management, access control, transaction handling. Always running — billed separately (free up to 10% of compute).
2. Query Processing Layer: virtual warehouses — temporary compute clusters you create, size, suspend, and resume. Pure CPU and RAM.
3. Storage Layer: compressed columnar micro-partition files stored in S3/Azure/GCS. Shared by all warehouses simultaneously.

Why separation matters: multiple warehouses can read the same data with zero contention. Compute can be paused when idle (billing stops). You scale compute and storage independently. This enables zero-copy cloning, multi-cluster concurrency, and per-second billing.

SQL Implementation

```
-- Three layers in practice
USE WAREHOUSE analytics_wh;      -- Query Processing layer
SELECT COUNT(*) FROM orders;     -- reads from Storage layer
-- Cloud Services optimized the plan automatically

-- Suspend warehouse = stop compute billing instantly
ALTER WAREHOUSE analytics_wh SUSPEND;
-- Storage data unchanged, Cloud Services still runs

-- Resume = wake up in ~1-2 seconds
ALTER WAREHOUSE analytics_wh RESUME;

-- Check all warehouses
SHOW WAREHOUSES;
```

KEY INSIGHT

Decoupled storage and compute = independent scaling, zero-copy cloning, no contention between concurrent workloads.

Q2 [Medium] [Architecture]

What is a micro-partition and what metadata does Snowflake maintain per partition?

Answer:

Micro-partitions are immutable compressed columnar files containing 50–500 MB of uncompressed data. Snowflake automatically creates and manages them — you never interact with them directly.

Metadata stored per partition: min value per column, max value per column, null count per column, distinct value count per column, and row count.

This metadata enables partition pruning: before reading any data, the optimizer checks min/max values to skip partitions that cannot contain matching rows. This is why a well-clustered 2TB table can answer a date-filtered query by reading only 0.1% of the data.

SQL Implementation

```
-- Partition pruning in action – check with EXPLAIN
EXPLAIN
SELECT SUM(amount) FROM orders WHERE order_date = '2024-03-15';
-- Look for: partitionsAssigned / partitionsTotal
-- Low ratio = good pruning = fast query

-- Add clustering key to improve pruning on large tables
ALTER TABLE orders CLUSTER BY (order_date);

-- Check clustering health
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(order_date)');
-- average_depth: < 3 = well clustered, > 5 = needs work

-- Metadata cache: serves min/max/count instantly (no warehouse)
SELECT MIN(order_date), MAX(order_date), COUNT(*) FROM orders;
```

KEY INSIGHT

Min/max metadata per partition enables pruning without reading data. Clustering co-locates similar values so fewer par

Q3 [Medium] [Architecture]

What is the difference between scaling up and scaling out in Snowflake?

Answer:

Scale UP: increase the warehouse size (XS → S → M → L → XL → 2XL → 4XL → 6XL). Each step doubles nodes, doubles cost per hour, and roughly doubles speed for compute/memory-bound queries. Fixes: slow individual queries, large sort operations, hash joins spilling to disk.

Scale OUT: add more clusters to a multi-cluster warehouse (Enterprise edition). Each cluster is an independent compute set. Fixes: query queue depth — many concurrent users waiting. Does nothing for individual query speed.

Diagnostic rule: Query Profile shows spill → scale UP. Task Manager shows queue → scale OUT.

SQL Implementation

```
-- Scale UP: single warehouse, bigger size
ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'XLARGE';

-- Scale OUT: multi-cluster for concurrency
ALTER WAREHOUSE analytics_wh
  SET MIN_CLUSTER_COUNT = 1
     MAX_CLUSTER_COUNT = 5
     SCALING_POLICY      = 'STANDARD'; -- add cluster when ANY queue
-- ECONOMY: wait longer before adding cluster (saves cost)

-- Monitor cluster utilization
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_LOAD_HISTORY
WHERE WAREHOUSE_NAME = 'ANALYTICS_WH'
  AND START_TIME > DATEADD('hour', -6, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;
```

KEY INSIGHT

Scale UP = faster queries. Scale OUT = more concurrent queries. Most engineers mix these up — the diagnostic is clear:

Q4 [Hard] [Architecture]

What are the three caching layers in Snowflake, what invalidates each, and what are the practical implications?

Answer:

1. Result Cache (Cloud Services): stores exact query results for 24 hours. No warehouse needed — free. Resets on every cache hit. Invalidated by: underlying data change, 24h expiry, any non-deterministic function (CURRENT_TIMESTAMP, RANDOM, UUID_STRING, CURRENT_DATE). Practical: identical dashboards run by 100 users in 24h hit cache after the first run.
2. Local Disk Cache (Warehouse SSD): micro-partitions read from S3 are cached on the warehouse SSD. Warm queries are dramatically faster. Invalidated by: warehouse suspend/resume (SSD is wiped on resume). Practical: do not aggressively suspend a warehouse mid-workload — you lose the warm cache and next query reads from S3.
3. Metadata Cache (Cloud Services): partition min/max, object metadata, COUNT(*). Always available. Never invalidated. Free. Serves SHOW commands, COUNT(*), MIN/MAX on partition keys.

SQL Implementation

```
-- Test result cache
SELECT COUNT(*) FROM orders;           -- first run: 30s
SELECT COUNT(*) FROM orders;           -- second run: 0ms = cache hit!

-- Bypass result cache for testing
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
ALTER SESSION SET USE_CACHED_RESULT = TRUE; -- re-enable

-- Non-deterministic = ALWAYS cache miss (avoid in dashboards)
SELECT COUNT(*), CURRENT_TIMESTAMP() FROM orders; -- always slow
SELECT COUNT(*), RANDOM() FROM orders;           -- always slow

-- Check if result cache was used
-- Query History → Execution Time = 0ms = result cache hit

-- Metadata cache: instant (no warehouse)
SELECT MIN(order_date), MAX(order_date), COUNT(*) FROM orders;
```

KEY INSIGHT

Result cache = 24h, free. Suspending warehouse clears SSD cache. CURRENT_TIMESTAMP in queries = result cache always mi

Q5 [Medium] [Architecture]

What is the difference between Time Travel and Fail-safe?

Answer:

Time Travel: user-accessible data recovery. You control it via SQL (AT, BEFORE, UNDROP). Duration: 0–1 day on Standard, up to 90 days on Enterprise. Configurable per object via DATA_RETENTION_TIME_IN_DAYS. Costs storage proportional to data change volume.

Fail-safe: Snowflake-managed emergency recovery. Always exactly 7 days. Begins AFTER Time Travel expires. NOT accessible via SQL — only Snowflake Support can recover data. Cannot be shortened or configured. Also costs storage.

GDPR note: setting DATA_RETENTION_TIME=0 BEFORE deletion prevents Time Travel from capturing the delete. But existing Time Travel data from before that change still persists.

SQL Implementation

```
-- Time Travel: query past state
SELECT * FROM orders AT (OFFSET => -3600); -- 1 hour ago
SELECT * FROM orders AT (
    TIMESTAMP => '2024-03-15 09:00:00'::TIMESTAMP_LTZ);
SELECT * FROM orders BEFORE (STATEMENT => '01abc...');

-- Recover a dropped table (within retention period)
UNDROP TABLE orders;
UNDROP SCHEMA my_schema;
UNDROP DATABASE my_db;

-- Set shorter retention on high-churn staging tables
ALTER TABLE stg_raw SET DATA_RETENTION_TIME_IN_DAYS = 1;
ALTER TABLE orders SET DATA_RETENTION_TIME_IN_DAYS = 30;

-- Transient tables: no Fail-safe (saves 7-day storage overhead)
CREATE TRANSIENT TABLE stg_orders (...);

-- Fail-safe: nothing you can run – call Snowflake Support
```

KEY INSIGHT

Time Travel = you control via SQL. Fail-safe = Snowflake controls, Support only. Both cost storage.

Q6 [Medium] [Architecture]

Explain zero-copy cloning, how it works internally, and its best use cases.

Answer:

Zero-copy cloning copies only the metadata (a pointer to existing micro-partitions). No data is physically duplicated at clone time. Source and clone share the same micro-partitions until one is modified — only changed partitions are then physically duplicated.

Initial storage cost = near zero. Divergence cost = only the delta between source and clone.

Best use cases:

- Instant dev/test environment from production (10TB prod = instant dev clone at ~zero cost)
- Pre-migration snapshot (clone → run risky operation → swap back if bad)
- Daily sandbox refresh for data scientists (zero-copy clone of gold layer)
- A/B testing transformation logic (two clones, two different transforms)

SQL Implementation

```
-- Clone entire database instantly (10TB = 1 second)
CREATE DATABASE dev_db CLONE prod_db;

-- Clone a table
CREATE TABLE orders_backup CLONE orders;

-- Clone at a point in time (Time Travel + Clone)
CREATE TABLE orders_march_end CLONE orders
    AT (TIMESTAMP => '2024-03-31 23:59:59'::TIMESTAMP_LTZ);

-- Pre-migration safety net
CREATE TABLE orders_pre_migration CLONE orders;
```

KEY INSIGHT

Clone = metadata copy. Divergence is what costs storage. Perfect for dev/test and pre-migration safety nets.

Q7 [Hard] [Architecture]**What Snowflake editions exist and what capabilities are gated to each?**

Answer:

Standard: core SQL, Time Travel up to 1 day, basic security (RBAC), all file formats.

Enterprise: adds Time Travel up to 90 days, multi-cluster warehouses (critical for concurrency), column-level security (masking policies), row access policies, materialized views, search optimization service, database replication.

Business Critical: adds HIPAA/PCI/SOC2 Type II compliance certifications, private link (AWS PrivateLink / Azure Private Link), database failover and failback, customer-managed keys (Tri-Secret Secure), higher encryption standards.

Virtual Private Snowflake (VPS): completely isolated Snowflake infrastructure on dedicated hardware. No shared resources with other Snowflake customers. Highest security isolation level.

SQL Implementation

```
-- Features gated by edition:

-- Enterprise required:
ALTER WAREHOUSE wh SET MIN_CLUSTER_COUNT=1 MAX_CLUSTER_COUNT=4; -- multi-cluster

CREATE MASKING POLICY email_mask ...; -- column-level security
CREATE ROW ACCESS POLICY region_pol ...; -- row-level security

CREATE MATERIALIZED VIEW mv_daily AS SELECT ...;

ALTER TABLE orders SET DATA_RETENTION_TIME_IN_DAYS = 30; -- up to 90 days

-- Business Critical required:
-- Private Link setup (done via AWS/Azure console)
-- CREATE FAILOVER GROUP ... -- database replication/failover

-- Check your current edition
SELECT CURRENT_ACCOUNT();
-- Admin → Account in Snowsight shows edition
```

KEY INSIGHT

Multi-cluster, masking, row access, 90-day Time Travel = Enterprise. HIPAA, private link, CMK = Business Critical.

Q8 [Hard] [Architecture]**What is Snowflake's transaction isolation level and what are its practical implications?**

Answer:

Snowflake uses Snapshot Isolation (SI). Every transaction sees a consistent snapshot of data as of its START time. This prevents: dirty reads, non-repeatable reads, phantom reads.

Practical implications:

1. Two sequential SELECT COUNT(*) queries CAN return different values if DML ran between them — both counts are correct for their respective snapshots.
2. Long-running transactions see stale data — the snapshot is from when the transaction began.
3. Write-write conflicts: if two sessions modify the same row, the last writer wins.

4. For consistent multi-statement reads, use explicit BEGIN...COMMIT to lock the snapshot.
5. Tasks and streams: stream offset does not advance until the consuming DML commits — this is how exactly-once semantics work.

SQL Implementation

```
-- Two queries returning different counts – BOTH correct
-- Session 1: SELECT COUNT(*) FROM orders; --> 1,000,000
-- (DELETE 500 rows runs between sessions)
-- Session 2: SELECT COUNT(*) FROM orders; --> 999,500
-- Both are correct for their snapshot time!

-- For consistent multi-statement reads:
BEGIN;
  SELECT COUNT(*) FROM orders;      -- snapshot locked here
  SELECT SUM(amount) FROM orders;    -- same snapshot guaranteed
COMMIT;

-- Investigate what DML ran between two queries
SELECT QUERY_ID, QUERY_TYPE, START_TIME, QUERY_TEXT
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TYPE IN ('DELETE', 'INSERT', 'UPDATE', 'MERGE')
  AND START_TIME BETWEEN '<time1>' AND '<time2>'
ORDER BY START_TIME;
```

KEY INSIGHT

Snapshot Isolation = each statement sees its own consistent view. Concurrent DML between statements = different result

DATA LOADING

Interview Questions — Data Loading

Q9 [Easy] [Data Loading]

What is the difference between internal and external stages? When would you choose each?

Answer:

Internal stages are managed by Snowflake in its own cloud storage. Three types:

- User stage (@~): automatically created per user, for personal use only
- Table stage (@%table_name): automatically created per table, files can only load into that table
- Named stage: explicitly created via CREATE STAGE, shareable across teams

External stages point to your own S3/Azure/GCS bucket. Data stays in your cloud storage — Snowflake reads it from there.

Choose external when: files already live in your cloud, other systems also read the same files, compliance requires you own all storage, or you want to use Snowpipe with existing S3 event infrastructure.

SQL Implementation

```
-- User stage (automatic per user)
PUT file:///data/orders.csv @~;
LIST @~;
```

KEY INSIGHT

Internal = Snowflake manages storage. External = your cloud, Snowflake reads it. Use external when compliance or exist

Q10 [Medium] [Data Loading]**How does Snowflake prevent duplicate loads in COPY INTO and how do you override this behavior?**

Answer:

Snowflake maintains a load history per stage+table combination for 64 days, tracking the ETag (file hash) of every successfully loaded file. Re-running COPY INTO for already-loaded files silently skips them — no error, no warning, no duplicates.

After 64 days, load history expires and files can be re-loaded. This is a common gotcha for long-running pipelines.

To override: use FORCE=TRUE. This ignores load history completely and re-loads all matching files. Risk: duplicates if you don't dedup afterward.

To check history: INFORMATION_SCHEMA.COPY_HISTORY table function or SNOWFLAKE.ACCOUNT_USAGE.LOAD_HISTORY view.

SQL Implementation

```
-- Normal run: skips already-loaded files automatically
COPY INTO orders FROM @s3_stage
  FILE_FORMAT = (FORMAT_NAME = 'csv_fmt');
-- Files loaded in the last 64 days are silently skipped

-- Force reload (WARNING: will create duplicates!)
COPY INTO orders FROM @s3_stage
  FILE_FORMAT = (FORMAT_NAME = 'csv_fmt')
  FORCE = TRUE;
-- Always dedup after FORCE=TRUE:
CREATE TABLE orders_deduped AS
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id ORDER BY _load_ts DESC
) = 1;

-- Check load history
SELECT FILE_NAME, ROW_COUNT, ERROR_COUNT, STATUS, LAST_LOAD_TIME
FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME => 'ORDERS',
  START_TIME => DATEADD('hour', -24, CURRENT_TIMESTAMP())
)) ORDER BY LAST_LOAD_TIME DESC;
```

KEY INSIGHT

64-day ETag-based dedup prevents re-loads automatically. FORCE=TRUE overrides but risks duplicates. History expires af

Q11 [Medium] [Data Loading]**Explain how Snowpipe AUTO_INGEST works end-to-end with AWS S3.**

Answer:

Step-by-step flow:

1. Create a pipe with AUTO_INGEST=TRUE — Snowflake creates a dedicated SQS queue and returns its ARN.
2. Get the SQS ARN from SHOW PIPES (notification_channel column).

3. Configure S3 Event Notification: ObjectCreated → SQS → paste the ARN.
4. When a new file lands in S3, S3 pushes an event to the SQS queue.
5. Snowflake's Snowpipe service polls SQS, detects the event, and runs the COPY INTO definition from the pipe.

Latency: 1–5 minutes from file arrival to data available in Snowflake.

Billing: serverless — billed per file/credit consumed, not per warehouse-hour.

Dedup: same 64-day ETag load history as regular COPY INTO.

SQL Implementation

```
-- Step 1: Create the pipe
CREATE PIPE orders_pipe
  AUTO_INGEST = TRUE
  ERROR_INTEGRATION = sns_alerts -- optional: alert on errors
AS
  COPY INTO orders
  FROM @s3_stage
  FILE_FORMAT = (FORMAT_NAME = 'csv_fmt');

-- Step 2: Get SQS ARN for S3 event notification
SHOW PIPES;
-- notification_channel column = SQS ARN to paste into AWS

-- Step 3: Configure in AWS
-- S3 bucket → Properties → Event Notifications
-- Add: All object create events → SQS → paste ARN

-- Step 4: Monitor pipe health
SELECT SYSTEM$PIPE_STATUS('orders_pipe');
-- Check: executionState=RUNNING, pendingFileCount

-- Step 5: Catch up after outage
ALTER PIPE orders_pipe REFRESH
  MODIFIED_AFTER = '2024-03-20T00:00:00';

-- Pause/resume
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;
```

KEY INSIGHT

AUTO_INGEST = S3 event → SQS queue → Snowpipe. Get SQS ARN from SHOW PIPES. Serverless billing.

Q12 [Medium] [Data Loading]

What is INFER_SCHEMA and how do you use it to auto-create a table from a staged file?

Answer:

INFER_SCHEMA reads a staged file (Parquet, ORC, Avro, JSON, or CSV with PARSE_HEADER=TRUE) and returns the detected schema as a result set with column names, data types, nullability, and ordinal position.

Use cases:

1. Validate file structure before committing to a load pipeline
2. Auto-create a table that exactly matches the file schema (USING TEMPLATE)
3. Use MATCH_BY_COLUMN_NAME in COPY INTO for name-based mapping instead of positional \$1,\$2,\$3

SQL Implementation

KEY INSIGHT *Infer schema of staged Parquet file*

INFER_SCHEMA + USING TEMPLATE = zero-code table creation from any staged file. MATCH_BY_COLUMN_NAME removes fragile po

Q13 [Easy] [Data Loading]

What is the difference between ON_ERROR=CONTINUE, ABORT_STATEMENT, and SKIP_FILE in COPY INTO?

Answer:

ABORT_STATEMENT (default): first bad row stops the entire COPY INTO. Zero rows loaded from any file if any file has an error. Use for strict zero-tolerance pipelines.

CONTINUE: skip individual bad rows, load all valid rows. Errors logged in COPY_HISTORY. Use for high-throughput pipelines with known data quality issues.

SKIP_FILE: if any row in a file errors, skip the ENTIRE file. Other files still load. Use when you need file-level atomicity — either all of a file loads or none of it does.

SKIP_FILE_n: skip file if it has more than n absolute error count.

SKIP_FILE_n%: skip file if error rate exceeds n%.

SQL Implementation

```
-- ABORT_STATEMENT (default): halt on first error
COPY INTO orders FROM @stage
  FILE_FORMAT = (FORMAT_NAME='csv_fmt');
  -- ON_ERROR = 'ABORT_STATEMENT' is implicit

-- CONTINUE: skip bad rows, load everything else
COPY INTO orders FROM @stage
  FILE_FORMAT = (FORMAT_NAME='csv_fmt')
  ON_ERROR      = 'CONTINUE';

-- SKIP_FILE: skip whole file if any error
COPY INTO orders FROM @stage
  FILE_FORMAT = (FORMAT_NAME='csv_fmt')
  ON_ERROR      = 'SKIP_FILE';

-- SKIP_FILE_5%: skip file if >5% of rows error
COPY INTO orders FROM @stage
  FILE_FORMAT = (FORMAT_NAME='csv_fmt')
  ON_ERROR      = 'SKIP_FILE_5%';

-- DRY RUN: see all errors without loading anything
COPY INTO orders FROM @stage
  FILE_FORMAT      = (FORMAT_NAME='csv_fmt')
  VALIDATION_MODE = 'RETURN_ALL_ERRORS';
```

KEY INSIGHT

ABORT=strictest. CONTINUE=most tolerant. SKIP_FILE=file-level atomicity. Always dry-run with VALIDATION_MODE first.

PERFORMANCE

Interview Questions — Performance

Q14 [Hard] [Performance]

What is query spilling, how do you diagnose it, and what are the fixes?

Answer:

Spilling occurs when a query operation (sort, hash join, aggregation) requires more memory than the warehouse nodes have. Data spills first to local SSD (10–100x slower than RAM), then to remote storage (S3/Azure/GCS — 1000x slower than RAM).

Diagnose: Query Profile in Snowsight UI shows "Bytes spilled to local storage" and "Bytes spilled to remote storage" operators. Also visible in ACCOUNT_USAGE.QUERY_HISTORY columns BYTES_SPILLED_TO_LOCAL_STORAGE and BYTES_SPILLED_TO_REMOTE_STORAGE.

Fixes in order of preference:

1. Rewrite query: SELECT only needed columns, filter before joining, reduce intermediate set size
2. Size up warehouse: more nodes = more total RAM per query
3. Break complex CTE into separate materialized steps

SQL Implementation

```
-- Diagnose spill in recent queries
SELECT
  QUERY_ID,
  TOTAL_ELAPSED_TIME/1000 AS secs,
  BYTES_SPILLED_TO_LOCAL_STORAGE/1e9 AS local_spill_gb,
  BYTES_SPILLED_TO_REMOTE_STORAGE/1e9 AS remote_spill_gb,
  LEFT(QUERY_TEXT, 200) AS query_preview
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME > DATEADD('day', -1, CURRENT_TIMESTAMP())
  AND (BYTES_SPILLED_TO_LOCAL_STORAGE > 0
       OR BYTES_SPILLED_TO_REMOTE_STORAGE > 0)
ORDER BY BYTES_SPILLED_TO_REMOTE_STORAGE DESC
LIMIT 10;

-- Fix 1: Avoid SELECT * (columnar storage -- fewer cols = less I/O)
-- BAD:  SELECT * FROM orders WHERE region='US';
-- GOOD: SELECT order_id, amount, date FROM orders WHERE region='US';

-- Fix 2: Size up warehouse
ALTER WAREHOUSE transform_wh SET WAREHOUSE_SIZE = 'XLARGE';

-- Fix 3: Materialize intermediate CTE
CREATE TEMPORARY TABLE tmp_filtered AS
  SELECT order_id, customer_id, amount
  FROM orders WHERE region = 'US';
```

KEY INSIGHT

Remote spill = warehouse too small. Local spill = borderline. Both dramatically slow queries. Check Query Profile first

Q15 [Hard] [Performance]

When should you add a clustering key and when should you NOT?

Answer:

ADD a clustering key when:

- Table is > 1 TB
- Queries consistently filter on the same column(s)
- Partition scan ratio is high (Query Profile shows 80%+ partitions scanned)

- Column has high cardinality (dates, IDs — not boolean or 3-value status)

Do NOT add when:

- Table is small (< 100 GB) — overhead exceeds benefit
- Query patterns are varied — clustering helps one filter pattern, hurts pruning for others
- Column has low cardinality (boolean, status with 5 values — all partitions overlap anyway)
- Table has extremely high write volume (Automatic Clustering maintenance becomes expensive)

Monitor clustering health after adding: `SYSTEM$CLUSTERING_INFORMATION` returns `average_depth` (target < 3).

SQL Implementation

```
-- Step 1: Check current clustering health
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(order_date)');
-- average_depth: ideally < 3
-- total_partition_count: use to estimate reclustering time

-- Step 2: Add clustering key
ALTER TABLE orders CLUSTER BY (order_date);

-- Composite clustering (for multi-column filters)
ALTER TABLE orders CLUSTER BY (order_date, region);

-- Expression-based clustering
ALTER TABLE orders CLUSTER BY (TO_DATE(created_at));

-- Step 3: Monitor reclustering progress
SELECT SYSTEM$CLUSTERING_INFORMATION('orders');
-- CLUSTERING_KEY_PROGRESS: 100% = done

-- Step 4: If patterns changed, drop the key
ALTER TABLE orders DROP CLUSTERING KEY;

-- Check partition scan ratio in EXPLAIN
EXPLAIN SELECT SUM(amount) FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
```

KEY INSIGHT

Clustering helps >1TB tables with consistent filter patterns on high-cardinality columns. Small tables and varied quer

Q16 [Medium] [Performance]

What is the QUALIFY clause and why is it the Snowflake-idiomatic way to filter on window functions?

Answer:

QUALIFY filters rows based on window function results without needing a subquery — analogous to HAVING for aggregate functions.

Without QUALIFY: you need an outer SELECT wrapping the window function calculation, adding one extra level of nesting.

With QUALIFY: filter directly in the same SELECT — cleaner, more readable, often more performant.

Most common patterns:

- Deduplication: `QUALIFY ROW_NUMBER() OVER (...) = 1`
- Top-N per group: `QUALIFY RANK() OVER (PARTITION BY group ORDER BY metric DESC) <= 3`

- First/last event per user
- Remove duplicate source rows before SCD2 MERGE

SQL Implementation

```
-- WITHOUT QUALIFY: needs subquery
SELECT * FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY order_date DESC
    ) AS rn
  FROM orders
) WHERE rn = 1;

-- WITH QUALIFY: same result, no subquery
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY order_date DESC
) = 1;

-- Top product per category (RANK)
SELECT product_id, category, SUM(revenue) AS rev
FROM sales
GROUP BY 1, 2
QUALIFY RANK() OVER (
  PARTITION BY category
  ORDER BY rev DESC
) = 1;

-- Dedup stream before SCD2 MERGE
SELECT * FROM cdc_stream
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY _load_ts DESC
) = 1;
```

KEY INSIGHT

QUALIFY = HAVING for window functions. Snowflake-specific, eliminates subquery nesting, essential for deduplication pa

Q17 [Hard] [Performance]

What is query acceleration service (QAS) and how does it differ from sizing up a warehouse?

Answer:

Query Acceleration Service is a serverless feature that offloads the scanning phase of certain queries to dynamically provisioned compute — specifically for queries with highly variable scan patterns.

Key difference from sizing up:

- Sizing up: bigger warehouse benefits ALL queries, you pay for it 24/7 even when not needed
- QAS: dynamically adds resources only for queries that need it, charged at a per-second serverless rate

Best for: warehouses where most queries are fast but occasional rogue queries scan much more data than average. QAS adds compute for those outliers without penalizing the normal workload with a bigger (more expensive) warehouse.

Not useful for: consistently large workloads that always need more compute — sizing up is cheaper there.

SQL Implementation

QAS = dynamic scale-out for outlier queries, pay per use. Sizing up = static, always-on cost. QAS for variable-intensi

Q18 [Medium] [Performance]**How do RANK, DENSE_RANK, and ROW_NUMBER differ? When do you use each?**

Answer:

All three assign a number to each row within a PARTITION ORDER BY window.

ROW_NUMBER: always unique sequential integers regardless of ties. 1, 2, 3, 4, 5. Best for deduplication (take exactly one row per key) and pagination.

RANK: tied rows share the same rank; the next rank SKIPS (creates gaps). 1, 2, 2, 4, 5. Best for leaderboards where gaps communicate the tie and how many competitors share the position.

DENSE_RANK: tied rows share the same rank; the next rank does NOT skip. 1, 2, 2, 3, 4. Best for tier/bracket assignment where you want sequential tier numbers despite ties.

SQL Implementation

```
SELECT customer_id, total_revenue,
       ROW_NUMBER() OVER (ORDER BY total_revenue DESC) AS row_num,
       -- 1, 2, 3, 4, 5 (always unique)
       RANK() OVER (ORDER BY total_revenue DESC) AS rnk,
       -- 1, 2, 2, 4, 5 (gaps after ties)
       DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS dense_rnk,
       -- 1, 2, 2, 3, 4 (no gaps after ties)
FROM customer_summary;

-- ROW_NUMBER: deduplication (take most recent per customer)
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY updated_at DESC
) = 1;

-- DENSE_RANK: assign spending quartiles (1-4, no gaps)
SELECT customer_id, total_spend,
       DENSE_RANK() OVER (ORDER BY total_spend DESC) AS tier
FROM customer_summary
QUALIFY tier <= 4; -- top 4 tiers
```

KEY INSIGHT

ROW_NUMBER=unique. RANK=gaps after ties. DENSE_RANK=no gaps. ROW_NUMBER for dedup, DENSE_RANK for brackets.

Q19 [Hard] [Performance]**How do you query nested JSON arrays in Snowflake using LATERAL FLATTEN?**

Answer:

Snowflake stores JSON natively in VARIANT columns. Colon notation (:) traverses JSON paths. LATERAL FLATTEN explodes arrays into rows — one row per array element.

The FLATTEN alias gives you:

- f.value: the array element itself
- f.index: 0-based position in the array
- f.key: key name (for object flattening)
- f.path: full path to the element

Chain multiple LATERAL FLATTENS for nested arrays. Use MODE='OBJECT' to flatten object keys.

SQL Implementation

```
-- Sample JSON: {"user_id":42,"items":[{"prod":"shoes","qty":2},{"prod":"hat","qty":1}]}

-- Query top-level fields
SELECT
  raw:user_id::NUMBER          AS user_id,
  raw:items[0]:prod::VARCHAR   AS first_item,  -- array index
  raw:meta.device::VARCHAR     AS device       -- dot notation
FROM events;

-- LATERAL FLATTEN: explode array into rows
SELECT
  e.event_id,
  f.index          AS item_position,
  f.value:prod::VARCHAR AS product,
  f.value:qty::NUMBER AS quantity
FROM events e,
LATERAL FLATTEN(INPUT => e.raw:items) f;

-- Flatten object keys into rows
SELECT f.key, f.value::VARCHAR
FROM events,
LATERAL FLATTEN(INPUT => raw:meta) f;

-- Nested flatten (array of arrays)
SELECT e.event_id, a.value::VARCHAR AS category, b.value::VARCHAR AS tag
FROM events e,
LATERAL FLATTEN(INPUT => e.raw:categories) a,
LATERAL FLATTEN(INPUT => a.value:tags) b;
```

KEY INSIGHT

LATERAL FLATTEN = UNNEST for Snowflake VARIANT. f.value=element, f.index=position, f.key=object key. Chain for nested

CDC & SCD

Interview Questions — CDC & SCD

Q20 [Easy] [CDC & SCD]

What is CDC and what are the four main patterns for implementing it in Snowflake?

Answer:

CDC (Change Data Capture) identifies and processes only rows that actually changed since the last pipeline run, instead of reloading the entire source table. For a 100M-row table with 0.1% daily changes, CDC processes 100K rows instead of 100M.

Four patterns:

1. Stream-based CDC: Snowflake Streams track every DML on Bronze tables automatically. Best native approach.
2. Hash-based CDC: compute MD5(all business columns) at load time, compare to last-known hash in a CDC_STATE control table. Works when source has no updated_at.

3. Timestamp watermark: process rows where updated_at > last successful run timestamp. Simple but misses deletes.
4. Log-based CDC: Debezium reads the source DB transaction log and sends events via Kafka to Snowflake. Lowest latency, highest source fidelity.

SQL Implementation

```
-- Hash-based CDC: detect changes without updated_at
COPY INTO bronze_customers (... , _src_row_hash)
FROM (
    SELECT ...,
        MD5(CONCAT_WS('|', $1,$2,$3,$4,$5,$6,$7)) AS _src_row_hash
    FROM @stage
);

-- Compare hashes to find changes
SELECT src.customer_id,
    CASE
        WHEN ctrl.last_hash IS NULL          THEN 'INSERT'
        WHEN ctrl.last_hash != src._src_row_hash THEN 'UPDATE'
        ELSE                                'NO_CHANGE'
    END AS cdc_operation
FROM bronze_customers src
LEFT JOIN cdc_state ctrl
    ON ctrl.business_key = src.customer_id
   AND ctrl.table_name = 'BRONZE_CUSTOMERS';

-- Stream-based CDC
CREATE STREAM orders_stream ON TABLE bronze_orders;
-- Streams track all DML automatically -- no hash needed
```

KEY INSIGHT

Snowflake Streams are the native CDC mechanism. Hash-based is the fallback for full-file extracts without updated_at.

Q21 [Medium] [CDC & SCD]

Explain METADATA\$ACTION and METADATA\$ISUPDATE in Snowflake streams with examples.

Answer:

Every stream row carries two system columns:

METADATA\$ACTION: INSERT (new row or the new value of an UPDATE) or DELETE (removed row or the old value of an UPDATE).

METADATA\$ISUPDATE: TRUE when the INSERT and DELETE rows are a paired UPDATE (not a net new insert or net delete). FALSE for actual inserts/deletes.

An UPDATE generates TWO stream rows:

- Row 1: old values, ACTION=DELETE, ISUPDATE=TRUE
- Row 2: new values, ACTION=INSERT, ISUPDATE=TRUE

To consume only new values of updates: filter ACTION=INSERT AND ISUPDATE=TRUE.

To consume only net-new inserts: filter ACTION=INSERT AND ISUPDATE=FALSE.

SQL Implementation

```
-- View stream contents with metadata columns
SELECT
```

KEY INSIGHT

UPDATE = DELETE(old)+INSERT(new), both with ISUPDATE=TRUE. Never consume just ACTION=INSERT without checking ISUPDATE

Q22 [Hard] [CDC & SCD]

What are all six SCD types? Give a brief description and use case for each.

Answer:

SCD Type 0 — Fixed/Retain: original value never changes. DIM_DATE — 2024-01-15 is always a Monday.

SCD Type 1 — Overwrite: UPDATE in place, old value lost. DIM_STORES — store renamed, old name gone forever.

SCD Type 2 — Add Row: expire old row (set EFF_END_DATE, IS_CURRENT=FALSE), insert new row. DIM_CUSTOMERS — customer segment changed, need full history.

SCD Type 3 — Add Column: add PREV_VALUE column alongside current. Only tracks one previous value. DIM_CUSTOMERS.PREV_SEGMENT.

SCD Type 4 — History Table: current values in main dim, all versions in a separate history table. DIM_PRODUCT_PRICE_HISTORY for fast current + full audit.

SCD Type 6 — Hybrid 1+2+3: combines all three. Type 2 versioning for tracked attributes + Type 1 overwrite for non-versioned cols + Type 3 PREV_ column. Most complete approach.

SQL Implementation

```
-- SCD0: Never touched after initial load
-- (no UPDATE statement ever runs on DIM_DATE)

-- SCD1: MERGE with full overwrite
MERGE INTO dim_stores tgt USING src s ON tgt.store_id=s.store_id
WHEN MATCHED THEN UPDATE SET tgt.store_name=s.store_name, ...
WHEN NOT MATCHED THEN INSERT ...;

-- SCD2: Three-step expire and insert
-- Step 1: Expire
UPDATE dim_customers SET is_current=FALSE, eff_end_date=CURRENT_DATE()-1
WHERE customer_id IN (SELECT customer_id FROM changed) AND is_current=TRUE;
-- Step 2: Insert new version
INSERT INTO dim_customers (... ,eff_start_date,eff_end_date,is_current,version_num)
SELECT ..., CURRENT_DATE(), NULL, TRUE, prev.version_num+1 FROM changed;

-- SCD3: PREV_SEGMENT column in dim (populated from expiring row)

-- SCD4: Write to separate history table on every price change
INSERT INTO dim_product_price_history(product_id,unit_price,valid_from)
SELECT product_id, unit_price, CURRENT_DATE() FROM changed_products;

-- SCD6: Combines all three in dim_customers
```

KEY INSIGHT

SCD0=never change. SCD1=overwrite. SCD2=full history rows. SCD3=one prev col. SCD4=history table. SCD6=hybrid 1+2+3.

Q23 [Hard] [CDC & SCD]

What is point-in-time FK resolution in SCD2 and why does it matter for fact tables?

Answer:

When loading a fact table that joins to an SCD2 dimension, you must find the dimension version that was ACTIVE when the business event occurred — not today's current version.

Wrong approach: JOIN ON is_current=TRUE. This always gives today's segment/price/etc, even for orders placed years ago under a different version.

Right approach: JOIN ON event_date BETWEEN eff_start_date AND COALESCE(eff_end_date, '9999-12-31'). This returns the SK of the version active when the order happened.

Why it matters: if a customer was in 'Consumer' segment for 2 years then moved to 'Corporate', their 2 years of Consumer-segment orders should NOT show as Corporate revenue. PIT resolution ensures correct historical attribution.

SQL Implementation

```
-- WRONG: gives today's segment for ALL historical orders
SELECT f.order_id, c.segment
FROM fact_orders f
JOIN dim_customers c
  ON c.customer_id = f.customer_id
 AND c.is_current = TRUE;      -- WRONG!
-- A customer who moved Corporate gets ALL old orders re-attributed

-- RIGHT: PIT lookup - find version active when order was placed
SELECT f.order_id, c.segment -- segment at time of order
FROM fact_orders f
JOIN dim_date d
  ON d.date_key = f.order_date_key
JOIN dim_customers c
  ON c.customer_id = f.customer_id
 AND d.full_date BETWEEN c.eff_start_date
   AND COALESCE(c.eff_end_date, '9999-12-31');

-- At fact LOAD time: store the PIT SK permanently
-- Future dim changes create new SKs, never touch old facts
-- SK in fact = permanent link to version active at event time
```

KEY INSIGHT

PIT lookup at fact load time = permanently correct historical attribution. IS_CURRENT=TRUE at query time = wrong for h

Q24 [Medium] [CDC & SCD]

When does a Snowflake stream offset advance and how does this enable exactly-once semantics?

Answer:

The stream offset advances ONLY when a DML transaction that consumes the stream commits successfully. A plain SELECT on the stream reads data but does NOT advance the offset.

If a task fails mid-execution (warehouse crash, query error, timeout), the DML hasn't committed, so the offset doesn't advance. The same data is available on the next run — this is the retry mechanism.

Exactly-once semantics: pair this with idempotent DML (MERGE instead of INSERT). Processing the same stream data twice with a MERGE on a natural key produces the same result both times — no duplicates.

Important: streams have a STALE_AFTER date visible in SHOW STREAMS. If you don't consume before this date, stream data is lost.

SQL Implementation

```
-- This does NOT advance the offset:
SELECT * FROM my_stream;  -- read only, offset stays
SELECT COUNT(*) FROM my_stream;
```

KEY INSIGHT

Offset advances on DML COMMIT only. Failed tasks = stream preserved = safe retry. MERGE = idempotent = exactly-once.

Q25 [Hard] [CDC & SCD]**How do you handle a soft delete in an SCD2 pipeline?**

Answer:

A soft delete means the source signals a record was deleted — but you NEVER physically delete rows from a data warehouse dimension. Physical deletion breaks fact FK resolution: any fact row with an SK pointing to the deleted dimension row becomes an orphan.

Soft delete pattern: when a stream row appears with METADATA\$ACTION=DELETE and METADATA\$ISUPDATE=FALSE, update the current dimension row to set IS_ACTIVE=FALSE (and optionally set a DELETED_AT timestamp). The row remains with IS_CURRENT=TRUE.

Historical facts still resolve correctly because the SK still exists. IS_ACTIVE=FALSE tells you the business state (customer left/account closed) without destroying the data.

SQL Implementation

```
-- Detect soft delete in stream
SELECT customer_id FROM cdc_streams.str_customers
WHERE METADATA$ACTION = 'DELETE'
  AND METADATA$ISUPDATE = FALSE; -- true delete, not update pair

-- Apply soft delete
UPDATE dim_customers tgt
SET
  is_active      = FALSE,
  deleted_at     = CURRENT_TIMESTAMP(),
  cdc_operation  = 'DELETE',
  scd_action     = 'SOFT_DELETE',
  dw_updated_at  = CURRENT_TIMESTAMP()
WHERE customer_id IN (
  SELECT customer_id FROM cdc_streams.str_customers
  WHERE METADATA$ACTION = 'DELETE'
    AND METADATA$ISUPDATE = FALSE
)
AND is_current = TRUE;

-- Query active customers only
SELECT * FROM dim_customers
WHERE is_current = TRUE AND is_active = TRUE;

-- Historical facts still resolve:
SELECT f.order_id, c.customer_name, c.is_active
FROM fact_orders f
JOIN dim_customers c ON c.sk_customer = f.sk_customer;
-- Even deleted customers still join correctly
```

KEY INSIGHT

NEVER physically delete from DW dimensions. Soft delete (IS_ACTIVE=FALSE) preserves FK integrity for all historical fa

Q26 [Hard] [CDC & SCD]**What is the difference between an append-only stream and a standard stream?**

Answer:

Standard stream: tracks all DML — INSERT, UPDATE (as DELETE+INSERT pair), and DELETE. More overhead internally. METADATA\$ACTION can be INSERT or DELETE. METADATA\$ISUPDATE differentiates

updates from true inserts/deletes.

Append-only stream: tracks ONLY INSERT operations. Cannot see updates or deletes — they are silently ignored. More efficient — Snowflake doesn't need to track deletion information.

Use append-only on: Bronze raw tables (COPY INTO only INSERTs, no updates or deletes happen there), event log tables, time-series append-only sources.

Use standard on: dimension tables, operational tables where updates and deletes occur.

SQL Implementation

```
-- Standard stream: full CDC
CREATE STREAM full_cdc_stream
  ON TABLE dim_customers;
-- Tracks: INSERTs, UPDATES (DELETE+INSERT pair), DELETES

-- Append-only: inserts only (more efficient)
CREATE STREAM insert_only_stream
  ON TABLE raw_orders
  APPEND_ONLY = TRUE;
-- Only sees: INSERT rows from COPY INTO or direct inserts
-- Ignores: DELETES, UPDATES
-- Use for: Bronze tables, event logs

-- Insert-only: for external tables
CREATE STREAM ext_stream
  ON EXTERNAL TABLE ext_orders
  INSERT_ONLY = TRUE;

-- Check stream type
SHOW STREAMS LIKE '%stream%';
-- MODE column: DEFAULT=standard, APPEND_ONLY, INSERT_ONLY

-- Append-only streams consume less storage overhead
-- and are faster to process since no delete tracking
```

KEY INSIGHT

Append-only = insert tracking only. More efficient for insert-only Bronze/log tables. Standard needed when source has

SECURITY

Interview Questions — Security

Q27 [Easy] [Security]

What four grants does a user need to run a SELECT query on a Snowflake table?

Answer:

Exactly four grants are required. Missing any one causes the query to fail:

1. GRANT USAGE ON DATABASE — allows the role to see the database exists
2. GRANT USAGE ON SCHEMA — allows the role to see the schema and its objects
3. GRANT SELECT ON TABLE — allows reading the table data
4. GRANT USAGE ON WAREHOUSE — allows using compute to execute the query

Common mistakes: granting SELECT on the table but forgetting USAGE on the database or schema. The error message says "insufficient privileges" but doesn't tell you which of the four is missing.

Best practice: add GRANT SELECT ON FUTURE TABLES IN SCHEMA so new tables are automatically accessible without re-granting.

SQL Implementation

```
-- All four required grants
GRANT USAGE ON DATABASE sales_db TO ROLE analyst_role;
GRANT USAGE ON SCHEMA sales_db.public TO ROLE analyst_role;
GRANT SELECT ON TABLE sales_db.public.orders TO ROLE analyst_role;
GRANT USAGE ON WAREHOUSE analytics_wh TO ROLE analyst_role;

-- Auto-grant for all future tables in schema
GRANT SELECT ON FUTURE TABLES
  IN SCHEMA sales_db.public TO ROLE analyst_role;

-- Auto-grant for all future views
GRANT SELECT ON FUTURE VIEWS
  IN SCHEMA sales_db.public TO ROLE analyst_role;

-- Verify what a role can do
SHOW GRANTS TO ROLE analyst_role;

-- Verify what a user has
SHOW GRANTS TO USER alice;

-- Verify who can access a specific table
SHOW GRANTS ON TABLE sales_db.public.orders;
```

KEY INSIGHT

DATABASE usage + SCHEMA usage + TABLE select + WAREHOUSE usage. All four required. FUTURE TABLES prevents re-granting.

Q28 [Medium] [Security]

What is the difference between a masking policy and a row access policy?

Answer:

Masking policy: applies to a COLUMN. Controls what VALUE the querier sees. The row is still returned, but the column shows masked/transformed data based on the querying role. The user doesn't know the real values exist.

Row access policy: applies to a TABLE (on a specific column used for filtering). Controls which ROWS are visible at all. Hidden rows are completely invisible — the user doesn't know they exist and cannot infer their presence.

They can be combined in layers: row access policy filters the user to their territory's rows, masking policy hides sensitive column values within those rows.

SQL Implementation

```
-- MASKING POLICY: hides column VALUE
CREATE MASKING POLICY email_mask
AS (v VARCHAR) RETURNS VARCHAR ->
CASE
  WHEN CURRENT_ROLE() = 'DATA_ENG_ROLE' THEN v -- full value
  WHEN CURRENT_ROLE() = 'ANALYST_ROLE'
    THEN CONCAT(LEFT(v,2), '***@', SPLIT_PART(v,'@',2)) -- partial
  ELSE '***@***.***' -- masked
```

KEY INSIGHT

Masking = hides column value, row still returned. Row access = hides entire rows. Both transparent to the querier.

Q29 [Hard] [Security]**What is EXECUTE AS OWNER vs EXECUTE AS CALLER in stored procedures?**

Answer:

EXECUTE AS CALLER (default): the stored procedure runs with the privileges of whoever CALLS it — their active role. If the caller doesn't have SELECT on a table, the procedure cannot query it. Safer — blast radius limited to caller's access.

EXECUTE AS OWNER: the procedure always runs with the privileges of the procedure OWNER's role (usually the role that created it — often SYSADMIN or a service role). The caller only needs USAGE on the procedure, not on all underlying objects.

Production rule: ETL/admin procedures that need elevated access → EXECUTE AS OWNER. User-facing procedures where caller's access should be enforced → EXECUTE AS CALLER.

SQL Implementation

```
-- EXECUTE AS CALLER (default) - runs with calling role
CREATE PROCEDURE sp_user_facing()
RETURNS STRING LANGUAGE SQL
EXECUTE AS CALLER -- caller must have all needed privileges
AS $$ SELECT COUNT(*) FROM orders; $$;

-- EXECUTE AS OWNER - runs with procedure owner's role
CREATE PROCEDURE sp_etl_elevated()
RETURNS STRING LANGUAGE SQL
EXECUTE AS OWNER -- caller only needs USAGE on procedure
AS $$ SELECT COUNT(*) FROM raw_db.raw.raw_orders; $$;
-- Even if caller is analyst with no Bronze access,
-- this procedure succeeds because OWNER has access

-- Check how a proc is defined
DESC PROCEDURE sp_etl_elevated();
-- Look for EXECUTE_AS column: OWNER or CALLER

-- DEBUG: if prod fails but dev works, check:
-- 1. Is it EXECUTE AS CALLER?
-- 2. What role is calling it in each environment?
-- 3. Does that role have the needed grants in prod?
```

KEY INSIGHT

CALLER = runs with calling role's privileges (safe). OWNER = runs with creator's privileges (elevated). ETL procs → OW

Q30 [Medium] [Security]**What is role inheritance in Snowflake and how does it affect privilege design?**

Answer:

When you GRANT ROLE child_role TO ROLE parent_role, the parent inherits ALL of the child's privileges. Any user with parent_role automatically has child_role's access without explicitly switching to child_role.

ACCOUNTADMIN inherits everything because both SYSADMIN and SECURITYADMIN are granted to it. This is why ACCOUNTADMIN should never be used for daily work — it is the maximum blast radius.

Design principle: build hierarchical role tiers where each tier adds capabilities:

- JUNIOR_ANALYST: Gold layer views only

- ANALYST: inherits JUNIOR + Silver read
- SENIOR_ANALYST: inherits ANALYST + Bronze read
- DATA_ENGINEER: inherits all + write access

SQL Implementation

```
-- Build role hierarchy
CREATE ROLE junior_analyst_role;
CREATE ROLE analyst_role;
CREATE ROLE senior_analyst_role;
CREATE ROLE data_engineer_role;

-- Inheritance chain
GRANT ROLE junior_analyst_role TO ROLE analyst_role;
GRANT ROLE analyst_role TO ROLE senior_analyst_role;
GRANT ROLE senior_analyst_role TO ROLE data_engineer_role;

-- Grant Gold to junior (all higher roles auto-inherit)
GRANT SELECT ON ALL VIEWS IN DATABASE gold_db
  TO ROLE junior_analyst_role;

-- Grant Silver to analyst level and above
GRANT SELECT ON ALL TABLES IN DATABASE silver_db
  TO ROLE analyst_role;

-- Show full privilege inheritance
SHOW GRANTS TO ROLE analyst_role; -- shows inherited too

-- Users
GRANT ROLE analyst_role TO USER bob;
-- Bob now has: Gold views + Silver tables (via inheritance)
```

KEY INSIGHT

Child granted to parent = parent inherits ALL child privileges. ACCOUNTADMIN = everything. Build tiers, assign users t

PIPELINES

Interview Questions — Pipelines

Q31 [Easy] [Pipelines]

Why are tasks always created in SUSPENDED state and what is the correct resume order for a DAG?

Answer:

Tasks are created SUSPENDED so you can build, connect, and verify the complete DAG before any execution begins. If the root task were auto-resumed, it would fire immediately — before child tasks are resumed — and skip all downstream steps on the first run.

Correct resume order: LEAF tasks first (tasks with no children), then work upward, ROOT task LAST.

Correct suspend order: ROOT task first (stops scheduling), then children.

EXECUTE TASK bypasses the schedule and manually triggers a task immediately — use for testing without waiting for the cron interval.

SQL Implementation

```
-- Resume: leaves first, root last. Suspend: root first. EXECUTE TASK for manual testing without waiting for schedule.
-- Create DAG (all created SUSPENDED automatically)
```

Q32 [Medium] [Pipelines]

What is WHEN SYSTEM\$STREAM_HAS_DATA and why should every stream-driven task use it?

Answer:

WHEN SYSTEM\$STREAM_HAS_DATA('stream_name') is a task condition that evaluates whether the stream has unconsumed data BEFORE starting the warehouse.

The evaluation is FREE — no warehouse compute needed. If the stream is empty, the task is skipped — the warehouse never wakes up, and you pay nothing for that task run.

Without this guard: a task running every 5 minutes wakes the warehouse 288 times per day. On a MEDIUM warehouse at 4 credits/hour, this wastes significant credits even on quiet periods.

With the guard: the warehouse only wakes when there is actually data to process. On a quiet Sunday with no source updates, zero credits are consumed.

SQL Implementation

```
-- WITHOUT guard: warehouse wakes every 5 min regardless
CREATE TASK wasteful_task
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
AS CALL sp_process();
-- 288 warehouse wakeups/day = 288 × 60s minimum billing

-- WITH guard: only wakes when stream has data
CREATE TASK efficient_task
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('bronze_db.raw.orders_stream')
AS CALL sp_process();
-- Evaluation is FREE. Warehouse only starts if data exists.

-- Multiple streams: use OR logic
CREATE TASK root_pipeline
  WAREHOUSE = etl_wh
  SCHEDULE = 'USING CRON 0 * * * * UTC' -- hourly
  WHEN
    SYSTEM$STREAM_HAS_DATA('str_customers') OR
    SYSTEM$STREAM_HAS_DATA('str_orders') OR
    SYSTEM$STREAM_HAS_DATA('str_products')
AS INSERT INTO audit_log VALUES (CURRENT_TIMESTAMP(), 'STARTED');
```

KEY INSIGHT

WHEN guard = free evaluation. Prevents warehouse wakeup when nothing changed. Critical for cost efficiency on stream-d

Q33 [Hard] [Pipelines]

What happens when a parent task in a DAG fails? How do you recover?

Answer:

When a task fails:

1. The failed task state = FAILED
2. All downstream child tasks state = SKIPPED (not FAILED)

3. The stream offset does NOT advance (the consuming DML never committed)
4. Stream data is preserved and available for the next run

Recovery options:

- Fix the root cause, then wait for the next scheduled run (stream data is still there)
- Or: EXECUTE TASK on the root to trigger immediately without waiting
- Or: EXECUTE TASK on the specific failed task to re-run from that midpoint

The stream preservation + idempotent MERGE = exactly-once semantics even after failures.

SQL Implementation

```
-- Check task state after failure
SELECT NAME, STATE, ERROR_CODE, ERROR_MESSAGE,
       SCHEDULED_TIME, COMPLETED_TIME,
       DATEDIFF('second', QUERY_START_TIME, COMPLETED_TIME) AS DURATION_SEC
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  SCHEDULED_TIME_RANGE_START => DATEADD('hour', -4, CURRENT_TIMESTAMP())
))
ORDER BY SCHEDULED_TIME DESC;

-- transform_task: FAILED
-- load_task:      SKIPPED (downstream)
-- notify_task:    SKIPPED (downstream)

-- Fix the stored procedure issue, then recover:

-- Option 1: Wait for next scheduled run
-- (stream data preserved, will auto-retry)

-- Option 2: Manual re-run from root
EXECUTE TASK root_task;

-- Option 3: Re-run from failed task only
EXECUTE TASK transform_task;

-- Monitor recovery
SELECT NAME, STATE FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  SCHEDULED_TIME_RANGE_START => DATEADD('minute', -15, CURRENT_TIMESTAMP())
)) ORDER BY SCHEDULED_TIME DESC;
```

KEY INSIGHT

Failed parent = downstream SKIPPED. Stream preserved = safe retry. EXECUTE TASK re-runs from any DAG point.

Q34 [Hard] [Pipelines]

What is the difference between a serverless task and a warehouse task?

Answer:

Warehouse task: uses a named virtual warehouse you create and manage. Subject to the 60-second minimum billing per resume. Performance is predictable and consistent. Best for: high-volume, complex transformations that consistently need specific compute.

Serverless task: USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE is a hint — Snowflake auto-sizes and auto-manages compute. Billed per-second of actual usage with no 60-second minimum. Snowflake automatically scales up or down based on historical execution patterns.

Serverless best for: variable-volume pipelines (sometimes 100 rows, sometimes 1M rows), very frequent tasks (every minute would waste 60s minimum per run), light tasks where the 60s minimum is wasteful.

SQL Implementation

KEY INSIGHT

Warehouse task: you manage the warehouse.
Serverless = pay per actual compute, no 60s minimum. Best for variable-volume or high-frequency tasks.

Q35 [Medium] [Pipelines]**What is a dynamic table and how does it differ from a stream+task pattern?**

Answer:

Dynamic tables are declarative incremental materialization. You write a SELECT query and specify TARGET_LAG (acceptable staleness). Snowflake automatically: detects what rows changed in the base tables, refreshes the result incrementally, manages the schedule, and handles dependency tracking.

Stream+task is imperative. You write the MERGE logic, manage the stream, set the schedule, handle errors, and wire up dependencies manually.

Dynamic tables are simpler for straightforward transformation pipelines. Stream+task gives you full control for complex conditional logic, multi-step pipelines, error handling, and CDC patterns with explicit SCD2 versioning.

SQL Implementation

```
-- DYNAMIC TABLE: declarative, Snowflake handles everything
CREATE DYNAMIC TABLE orders_summary
  TARGET_LAG = '15 minutes' -- refresh within 15 min of source change
  WAREHOUSE = analytics_wh
AS
  SELECT
    customer_id,
    region,
    DATE_TRUNC('month', order_date) AS order_month,
    SUM(amount) AS monthly_revenue,
    COUNT(*) AS order_count
  FROM orders
  GROUP BY 1,2,3;
-- Snowflake automatically detects changes in orders
-- and refreshes orders_summary incrementally

-- Manual refresh if needed
ALTER DYNAMIC TABLE orders_summary REFRESH;

SHOW DYNAMIC TABLES;
-- DATA_TIMESTAMP: shows when data was last refreshed

-- STREAM+TASK: imperative, you write everything
CREATE STREAM orders_stream ON TABLE orders;
CREATE TASK refresh_task WAREHOUSE=etl_wh SCHEDULE='15 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
AS MERGE INTO orders_summary ...; -- you write this
```

KEY INSIGHT

Dynamic tables = declarative (write SELECT, Snowflake manages refresh). Stream+task = imperative (you write MERGE, man

SQL

Interview Questions — SQL

Q36 [Hard] [SQL]

How do you implement sessionization of user events in Snowflake SQL?

Answer:

Sessionization groups events into sessions based on an inactivity gap threshold (e.g., 30 minutes of idle = new session).

Pattern:

1. Use LAG to get the previous event timestamp per user
2. Check if the gap exceeds the threshold → flag as new session start (IS_NEW_SESSION=1)
3. Use SUM(IS_NEW_SESSION) OVER (PARTITION BY user ORDER BY ts ROWS UNBOUNDED PRECEDING) as a cumulative counter → generates unique session IDs

SQL Implementation

```
WITH events_with_gap AS (
  SELECT
    user_id,
    event_ts,
    event_type,
    LAG(event_ts) OVER (
      PARTITION BY user_id
      ORDER BY event_ts
    ) AS prev_event_ts,
    -- Flag: is this event the start of a new session?
    CASE
      WHEN DATEDIFF('minute',
        LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts),
        event_ts) > 30
      THEN 1 -- gap > 30 min = new session
      WHEN LAG(event_ts) OVER (
        PARTITION BY user_id ORDER BY event_ts) IS NULL
      THEN 1 -- first event ever = new session
      ELSE 0
    END AS is_new_session
  FROM raw_events
),
-- Cumulative sum creates unique session IDs!
sessions AS (
  SELECT *,
    SUM(is_new_session) OVER (
      PARTITION BY user_id
      ORDER BY event_ts
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS session_id
  FROM events_with_gap
)
-- Final aggregation
SELECT
  user_id,
  session_id,
  MIN(event_ts) AS session_start,
  MAX(event_ts) AS session_end,
  COUNT(*) AS event_count,
  DATEDIFF('second', MIN(event_ts), MAX(event_ts)) AS session_duration_sec
FROM sessions
GROUP BY 1, 2
ORDER BY user_id, session_start;
```

KEY INSIGHT

SUM(is_new_session) OVER (ROWS UNBOUNDED PRECEDING) = elegant cumulative session ID counter. LAG detects the gap.

Q37 [Medium] [SQL]

How do you build a 7-day rolling average and a running YTD total using window functions?

Answer:

Window frame clauses control which rows are included in the window calculation:

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW: exactly 7 rows (current + 6 before)

RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW: value-based, handles date gaps

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW: all rows from start of partition to current

SQL Implementation

```
-- 7-day rolling average
SELECT
  full_date,
  daily_revenue,
  AVG(daily_revenue) OVER (
    ORDER BY full_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW -- 7 rows exactly
  ) AS rolling_7d_avg,
  -- Exclude current day from rolling (trailing only):
  AVG(daily_revenue) OVER (
    ORDER BY full_date
    ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING -- 7 rows before today
  ) AS trailing_7d_avg
FROM daily_sales;

-- Running YTD total (resets each year)
SELECT
  full_date,
  year,
  daily_revenue,
  SUM(daily_revenue) OVER (
    PARTITION BY year -- reset at year boundary
    ORDER BY full_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS ytd_revenue
FROM daily_sales;

-- Month-over-month growth rate
SELECT
  year, month_num, monthly_revenue,
  LAG(monthly_revenue,1) OVER (ORDER BY year, month_num) AS prev_month,
  ROUND(
    (monthly_revenue - LAG(monthly_revenue,1) OVER (ORDER BY year, month_num))
    / NULLIF(LAG(monthly_revenue,1) OVER (ORDER BY year, month_num),0) * 100
    , 2) AS mom_growth_pct
FROM monthly_sales;
```

KEY INSIGHT

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7-row rolling. PARTITION BY year + UNBOUNDED PRECEDING = YTD that resets each year.

Q38 [Hard] [SQL]

How do you deduplicate a large Snowflake table efficiently using QUALIFY?

Answer:

QUALIFY with ROW_NUMBER() is the Snowflake-idiomatic deduplication pattern. It eliminates the need for a subquery, is readable, and lets the optimizer push the window function calculation efficiently.

For large tables: consider creating a new deduplicated table and swapping, rather than DELETE with a subquery (DELETE with subquery can be slow on billions of rows as it processes row by row).

When to use DELETE vs CREATE TABLE AS SELECT: use CTAS for very large tables — it parallelizes better. Use DELETE for smaller dedup operations.

SQL Implementation

```
-- QUALIFY approach (most readable, no subquery)
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id      -- dedup key
  ORDER BY updated_at DESC   -- keep most recent
) = 1;

-- Create deduplicated table (for large tables)
CREATE TABLE orders_deduped AS
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id
  ORDER BY updated_at DESC
) = 1;

-- Verify counts
SELECT COUNT(*) FROM orders;      -- original
SELECT COUNT(*) FROM orders_deduped; -- deduped

-- Swap tables
ALTER TABLE orders      RENAME TO orders_with_dupes;
ALTER TABLE orders_deduped RENAME TO orders;

-- Drop backup after verification
DROP TABLE orders_with_dupes;

-- Alternative: dedup at query time without new table
SELECT order_id, customer_id, amount, order_date
FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id ORDER BY updated_at DESC
) = 1;
```

KEY INSIGHT

QUALIFY ROW_NUMBER() = Snowflake-idiomatic dedup. For large tables: CTAS + swap is faster than DELETE subquery.

COST

Interview Questions — Cost

Q39 [Easy] [Cost]

How is Snowflake warehouse billing calculated?

Answer:

Billing is per-second with a 60-second minimum on each resume. If a warehouse runs for 5 seconds, you pay for 60. If it runs for 90 seconds, you pay for 90.

Credit consumption by size: XS=1/hr, S=2/hr, M=4/hr, L=8/hr, XL=16/hr, 2XL=32/hr.

Each resume is a new 60-second minimum. A warehouse that runs a 2-second query every minute: pays 60 seconds $\times$ (2 credits/hour / 3600) $\times$ 1440 minutes = much more than necessary.

Key levers: AUTO_SUSPEND (set to 60s for most workloads), AUTO_RESUME=TRUE (1-2 second startup latency), right-size warehouses for their actual workload.

SQL Implementation

```
-- Create warehouse with optimal cost settings
CREATE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE = 'MEDIUM'    -- 4 credits/hour
  AUTO_SUSPEND    = 60          -- suspend after 60s idle
  AUTO_RESUME     = TRUE;       -- wake automatically on query

-- Billing math example:
-- MEDIUM = 4 credits/hr = 0.001111 credits/second
-- A query that runs 45 seconds:
-- max(60s, 45s) = 60s minimum = 60  $\times$  0.001111 = 0.0667 credits
-- A query that runs 3 minutes:
-- 180s  $\times$  0.001111 = 0.2 credits

-- Check daily credit consumption
SELECT USAGE_DATE,
       SUM(CREDITS_USED_COMPUTE)      AS compute_credits,
       SUM(CREDITS_USED_CLOUD_SERVICES) AS cloud_svc_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE USAGE_DATE >= DATE_TRUNC('month', CURRENT_DATE())
GROUP BY 1 ORDER BY 1 DESC;

-- Warehouse with no AUTO_SUSPEND runs 24/7
SHOW WAREHOUSES;
-- AUTO_SUSPEND = 0 means NEVER suspends -- fix immediately!
```

KEY INSIGHT

60-second minimum per resume. AUTO_SUSPEND=60 stops idle billing. A warehouse with AUTO_SUSPEND=0 bills 24/7 even with

Q40 [Medium] [Cost]

What are the three main Snowflake cost components and how do you reduce each?

Answer:

1. Compute (largest driver): warehouse credits per second $\times$ size. Reduce by: AUTO_SUSPEND=60, right-sizing (don't use LARGE for COPY INTO which is I/O-bound), serverless for variable workloads, result cache (same query = 0 credits).
2. Storage: compressed active data + Time Travel data + Fail-safe data. Reduce by: TRANSIENT tables for staging (no 7-day Fail-safe), shorter DATA_RETENTION_TIME on volatile tables, compress source files before loading.
3. Data Transfer: egress between cloud regions or providers. Zero within same cloud region. Reduce by: co-locate Snowflake and source/destination systems in same region, avoid cross-region Data Sharing.

SQL Implementation

```
-- Identify storage cost by layer
SELECT
  TABLE_CATALOG AS db,
  TABLE_NAME,
  ROUND(ACTIVE_BYTES/1e9, 2)      AS active_gb,
  ROUND(TIME_TRAVEL_BYTES/1e9, 2) AS time_travel_gb, -- cost here!
```

KEY INSIGHT

Compute = biggest driver. Storage = Time Travel + Fail-safe surprise. TRANSIENT tables eliminate Fail-safe for staging

Q41 [Hard] [Cost]**What is the cloud services credit threshold and what causes overage?**

Answer:

Cloud services (metadata, query optimization, auth) are free up to 10% of daily compute credits. Above 10%, you are billed at the same credit rate as compute. This is a per-day, per-account threshold.

Common causes of cloud services overage:

1. SHOW commands in loops — BI tool schema introspection on every dashboard refresh
2. Non-deterministic functions (CURRENT_TIMESTAMP, RANDOM) in queries — bypass result cache, force full re-parse and planning every time
3. JDBC/ODBC applications polling with high-frequency short queries
4. Excessive INFORMATION_SCHEMA queries per minute

Fix: replace SHOW commands with INFORMATION_SCHEMA queries (which use the result cache), remove non-deterministic functions from frequently-run queries, batch short queries.

SQL Implementation

```
-- Find what's consuming cloud services credits
SELECT
  USER_NAME,
  QUERY_TYPE,
  COUNT(*)                AS query_count,
  SUM(CREDITS_USED_CLOUD_SERVICES) AS cloud_credits,
  AVG(TOTAL_ELAPSED_TIME)/1000    AS avg_sec
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATE_TRUNC('month', CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 4 DESC LIMIT 20;

-- Look for: high SHOW command count, non-deterministic functions

-- Find SHOW commands (heavy metadata)
SELECT USER_NAME, COUNT(*), SUM(CREDITS_USED_CLOUD_SERVICES)
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TYPE = 'SHOW'
  AND START_TIME >= DATE_TRUNC('month', CURRENT_TIMESTAMP())
GROUP BY 1 ORDER BY 3 DESC;

-- FIX: Replace SHOW with INFORMATION_SCHEMA (uses result cache)
-- Instead of: SHOW TABLES IN DATABASE mydb
SELECT * FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_CATALOG = 'MYDB'; -- cached for 24h!
```

KEY INSIGHT

Cloud services > 10% = extra billing. SHOW commands in BI tool refresh loops are #1 cause. Replace with INFORMATION_SC

Q42 [Hard] [Cost]**How do resource monitors work and what are the four trigger action options?**

Answer:

Resource monitors track credit usage against a CREDIT_QUOTA per FREQUENCY (daily, weekly, monthly, yearly, never-reset). They apply to an account or specific warehouses.

Four trigger actions (set at percentage thresholds):

1. NOTIFY: sends an email alert to account admins — no action taken on the warehouse
2. NOTIFY & SUSPEND: sends email AND suspends the warehouse after current queries finish
3. NOTIFY & SUSPEND_IMMEDIATE: sends email AND kills ALL running queries immediately

You can stack multiple triggers at different percentages (e.g., NOTIFY at 75%, SUSPEND at 100%, SUSPEND_IMMEDIATE at 115%). Account-level monitors catch all warehouses; warehouse-level enables per-team cost attribution.

SQL Implementation

```
-- Create resource monitor with escalating triggers
CREATE RESOURCE MONITOR monthly_cap
  CREDIT_QUOTA      = 2000          -- 2000 credits per month
  FREQUENCY         = MONTHLY
  START_TIMESTAMP   = IMMEDIATELY
  TRIGGERS
    ON 50 PERCENT DO NOTIFY          -- early warning
    ON 75 PERCENT DO NOTIFY          -- escalation warning
    ON 90 PERCENT DO NOTIFY          -- critical warning
    ON 100 PERCENT DO SUSPEND        -- soft stop
    ON 115 PERCENT DO SUSPEND_IMMEDIATE; -- hard stop

-- Account-level (catches all warehouses)
ALTER ACCOUNT SET RESOURCE_MONITOR = monthly_cap;

-- Warehouse-level (per-team cost control)
ALTER WAREHOUSE analytics_wh SET RESOURCE_MONITOR = monthly_cap;
ALTER WAREHOUSE etl_wh      SET RESOURCE_MONITOR = monthly_cap;

-- Check current usage against quota
SHOW RESOURCE MONITORS;

-- Credit usage this month
SELECT USAGE_DATE, CREDITS_USED_COMPUTE
FROM SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE USAGE_DATE >= DATE_TRUNC('month', CURRENT_DATE())
ORDER BY USAGE_DATE;
```

KEY INSIGHT

SUSPEND lets running queries finish. SUSPEND_IMMEDIATE kills them. Stack both at different thresholds for graduated re

SECTION 2

Scenario-Based Interview Questions — 50 Real-World Problems

Scenario-based questions reveal whether you have actually operated Snowflake in production. Interviewers present realistic workplace situations and expect a structured approach: diagnose first → fix → prevent recurrence. Each answer includes the approach, SQL, and key insight.

PERFORMANCE

Scenario Questions — Performance

Scenario 1 [Hard] [Performance]

SITUATION:

Your analytics dashboard queries went from 3 seconds to 8 minutes after a fact table grew from 200 GB to 4 TB over 8 months. No other changes were made to the pipeline.

QUESTION:

Walk through your complete diagnosis and resolution.

APPROACH:

Step 1: Open Query Profile for a slow query — check "Partitions scanned / Partitions total". If 90%+ = partition pruning failure due to data growth mixing partition ranges.

Step 2: Run `SYSTEM$CLUSTERING_INFORMATION`. If `average_depth > 4` = poorly clustered.

Step 3: Identify the filter column(s) every slow dashboard query uses — almost certainly a date column.

Step 4: Add clustering key on that column and monitor reclustering progress.

Step 5: Check for Bytes spilled — if high, warehouse also needs sizing up.

SQL Implementation

```
-- Step 1: Check partition scan ratio
EXPLAIN
SELECT SUM(amount) FROM fact_orders
WHERE order_date = '2024-03-15';
-- partitionsAssigned / partitionsTotal: should be < 5%

-- Step 2: Clustering health check
SELECT SYSTEM$CLUSTERING_INFORMATION('fact_orders', '(order_date)');
-- average_depth > 3 = poorly clustered

-- Step 3: Add clustering key
ALTER TABLE fact_orders CLUSTER BY (order_date);

-- Step 4: Monitor reclustering (background process)
SELECT SYSTEM$CLUSTERING_INFORMATION('fact_orders');
-- CLUSTERING_KEY_PROGRESS: 0%→100% over hours/days

-- Step 5: Check for spill
SELECT BYTES_SPILLED_TO_REMOTE_STORAGE/1e9 AS remote_spill_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TEXT ILIKE '%fact_orders%'
AND START_TIME > DATEADD('day', -1, CURRENT_TIMESTAMP())
ORDER BY TOTAL_ELAPSED_TIME DESC LIMIT 5;

-- If spill is high: size up warehouse
ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'XLARGE';
```

KEY INSIGHT

Table growth mixes insertion-order data across date ranges, destroying partition pruning. Clustering key reorganizes s

Scenario 2 [Medium] [Performance]

SITUATION:

The same query returns in 0ms sometimes and 30 seconds other times on the same warehouse. Data has not changed and no other users are running queries.

QUESTION:

Is this a bug? Explain exactly what is happening.

APPROACH:

This is not a bug — it is the result cache working correctly. The 0ms runs are result cache hits: Snowflake returns a cached result with zero warehouse compute. The 30-second runs are cache misses (first run after 24h expiry, or after underlying data changed). This is expected behavior and a major feature of Snowflake. However, if the query contains `CURRENT_TIMESTAMP()`, `RANDOM()`, or other non-deterministic functions, it will ALWAYS miss the result cache.

SQL Implementation

```
-- Test: check if result cache is being used
SELECT COUNT(*) FROM orders;
-- Run again immediately: 0ms = result cache hit

-- Non-deterministic functions ALWAYS bypass result cache:
SELECT COUNT(*), CURRENT_TIMESTAMP() FROM orders; -- always 30s
SELECT COUNT(*), RANDOM() FROM orders;           -- always 30s
SELECT COUNT(*), UUID_STRING() FROM orders;       -- always 30s

-- Verify in Query History
-- Execution Time = 0ms --> result cache hit
-- Execution Time > 0ms --> actual compute run

-- Disable result cache for testing
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
ALTER SESSION SET USE_CACHED_RESULT = TRUE; -- re-enable

-- The result cache resets its 24h timer on every hit
-- So frequently-run queries stay cached indefinitely
```

KEY INSIGHT

Result cache = 24h, free, no warehouse. Non-deterministic functions always bypass it. Variable execution time is expected.

Scenario 3 [Hard] [Performance]**SITUATION:**

200 concurrent BI dashboard users are reporting slow query times on your analytics warehouse. You already sized the warehouse to XL. Sizing to 2XL did not help.

QUESTION:

Why did sizing to 2XL not solve the problem and what is the correct fix?

APPROACH:

Scaling UP (bigger size) gives more CPU and RAM per query — it helps individual slow queries, not concurrent user queues. 200 users queuing means the warehouse has a concurrency problem, not a per-query computation problem. The correct fix is scaling OUT: enable multi-cluster warehouse. Each cluster handles an independent pool of queries. With 4 clusters, 4× the concurrent throughput without any queue.

SQL Implementation

```
-- Scale UP helped nothing because queries are queuing, not slow
```

KEY INSIGHT

Scale UP = faster individual queries. Scale OUT = more concurrent queries. 200 users queuing = concurrency problem = s

DATA LOADING

Scenario Questions — Data Loading

Scenario 4 [Hard] [Data Loading]

SITUATION:

Snowpipe has been working for 4 months. This morning files are landing in S3 but nothing is being loaded into Snowflake. No errors appear in the Snowflake UI.

QUESTION:

Step-by-step: how do you diagnose and recover?

APPROACH:

Step 1: SYSTEM\$PIPE_STATUS — check executionState, pendingFileCount, notificationChannelStatus. Step 2: Check if pipe was accidentally paused (SHOW PIPES). Step 3: Go to AWS side — was the S3 event notification modified, SQS queue deleted, or IAM role permissions revoked? Step 4: Check Snowflake load history for error messages. Step 5: REFRESH pipe to catch up on missed files.

SQL Implementation

```
-- Step 1: Check pipe health
SELECT SYSTEM$PIPE_STATUS('orders_pipe');
-- executionState: RUNNING=healthy, PAUSED=problem
-- notificationChannelStatus: connected/disconnected
-- pendingFileCount: files waiting to process

-- Step 2: Check if accidentally paused
SHOW PIPES LIKE 'orders_pipe';
-- If paused:
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;

-- Step 3: Check load history for errors
SELECT *
FROM TABLE(INFORMATION_SCHEMA.PIPES_USAGE_HISTORY(
    DATE_RANGE_START => DATEADD('day', -1, CURRENT_TIMESTAMP()),
    PIPE_NAME => 'ORDERS_PIPE'
));

-- Step 4: Catch up on ALL missed files
ALTER PIPE orders_pipe REFRESH
    MODIFIED_AFTER = '2024-03-20T00:00:00';

-- Step 5: Verify loading resumed
SELECT FILE_NAME, STATUS, LAST_LOAD_TIME
FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
    TABLE_NAME => 'ORDERS',
    START_TIME => DATEADD('hour', -1, CURRENT_TIMESTAMP())
));
```

KEY INSIGHT

Most common: SQS ARN changed (S3 event notification modified), IAM permissions revoked, pipe paused. REFRESH catches u

Scenario 5 [Medium] [Data Loading]

SITUATION:

A junior developer accidentally ran `COPY INTO orders` with `FORCE=TRUE`, creating 500,000 duplicate rows. The mistake happened 2 hours ago.

QUESTION:

What is the fastest way to fix this?

APPROACH:

Time Travel is the fastest recovery — 2 hours is well within the retention window. Find the `query_id` of the bad `COPY INTO` from `QUERY_HISTORY`, clone the table to the state `BEFORE` that statement, verify the clone, then swap tables. This is safer and faster than writing a dedup query on 500K + original rows.

SQL Implementation

```
-- Step 1: Find the bad COPY INTO query_id
SELECT QUERY_ID, QUERY_TEXT, START_TIME
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TEXT ILIKE '%COPY INTO orders%FORCE%'
  AND START_TIME > DATEADD('hour', -3, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;

-- Step 2: Clone table to state BEFORE the bad COPY
CREATE TABLE orders_clean CLONE orders
  BEFORE (STATEMENT => '<query_id_of_bad_copy>');

-- Step 3: Verify row count is correct
SELECT COUNT(*) FROM orders_clean;
-- Should match pre-duplication count

-- Step 4: Swap tables
ALTER TABLE orders      RENAME TO orders_with_dupes;
ALTER TABLE orders_clean RENAME TO orders;

-- Step 5: Verify
SELECT COUNT(*) FROM orders;

-- Step 6: Drop backup after confirming
DROP TABLE orders_with_dupes;

-- Alternative if Time Travel not available:
CREATE TABLE orders_fixed AS
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id ORDER BY _load_ts DESC
) = 1;
```

KEY INSIGHT

Time Travel + zero-copy clone = instant recovery for recent mistakes. Always faster than dedup SQL for large duplicati

CDC & SCD

Scenario Questions — CDC & SCD

Scenario 6 [Hard] [CDC & SCD]

SITUATION:

Your `dim_customers` SCD2 table suddenly has 847 customers with two `IS_CURRENT=TRUE` rows each. Analysts are seeing doubled revenue in Gold dashboards.

QUESTION:

What caused this and how do you fix it immediately and permanently?

APPROACH:

Root cause: race condition — two instances of the SCD2 task ran concurrently (or nearly so). Both read the stream, both saw no current record for those customers, and both inserted new current rows before the other committed. The stream can be consumed by two concurrent sessions in some edge cases.

Immediate fix: identify duplicates, delete extras keeping the latest version. Permanent fix: add `QUALIFY` to deduplicate the stream source before the `MERGE` in the stored procedure.

SQL Implementation

```
-- Step 1: Identify duplicates
SELECT customer_id, COUNT(*) AS cnt
FROM dim_customers
WHERE is_current = TRUE
GROUP BY 1
HAVING cnt > 1;

-- Step 2: Remove duplicates – keep row with latest eff_start_date
DELETE FROM dim_customers
WHERE sk_customer IN (
    SELECT sk_customer FROM (
        SELECT sk_customer,
            ROW_NUMBER() OVER (
                PARTITION BY customer_id
                ORDER BY eff_start_date DESC, dw_created_at DESC
            ) AS rn
        FROM dim_customers
        WHERE is_current = TRUE
    ) WHERE rn > 1
);

-- Step 3: Verify no more duplicates
SELECT customer_id, COUNT(*) FROM dim_customers
WHERE is_current = TRUE
GROUP BY 1 HAVING COUNT(*) > 1;

-- Step 4: Fix stored procedure – QUALIFY stream before MERGE
-- In SP_SCD2_CUSTOMERS, add to the source CTE:
SELECT ...
FROM cdc_db.streams.str_customers
WHERE METADATA$ACTION = 'INSERT'
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY _load_ts DESC
) = 1; -- dedup BEFORE merging into Silver
```

KEY INSIGHT

`QUALIFY` before `MERGE` deduplicates stream source and prevents race-condition duplicates. Snowflake `UNIQUE` constraints a

Scenario 7 [Hard] [CDC & SCD]**SITUATION:**

Revenue reports show customers with their current segment instead of their segment at the time of purchase. Your SCD2 dim_customers table has correct history with multiple versions per customer.

QUESTION:

What is wrong with the fact loading procedure and how do you fix it?

APPROACH:

Classic point-in-time FK resolution bug. The fact table was loaded using IS_CURRENT=TRUE join instead of a PIT lookup. Every fact row has the SK of today's customer version, not the version active when the order was placed. You need to re-derive SK_CUSTOMER in the fact table using the correct BETWEEN join.

SQL Implementation

```
-- The BUG in the fact loading procedure:
INSERT INTO fact_orders
SELECT o.*, c.sk_customer
FROM raw_orders o
JOIN dim_customers c
  ON c.customer_id = o.customer_id
 AND c.is_current = TRUE; -- WRONG! Gets today's version for ALL orders

-- Verify the bug: customer who changed segment should show different
-- segments for orders before and after the change
SELECT f.order_id, f.order_date_key, c.segment
FROM fact_orders f
JOIN dim_customers c ON c.sk_customer = f.sk_customer
WHERE c.customer_id = 'C001'
ORDER BY f.order_date_key;
-- If all rows show same segment = bug confirmed

-- THE FIX: Point-in-time lookup
CREATE TABLE fact_orders_corrected AS
SELECT
  o.order_id,
  c.sk_customer,      -- version active WHEN order was placed
  o.order_date_key,
  o.total_amount
FROM raw_orders o
JOIN dim_date d ON d.date_key = o.order_date_key
JOIN dim_customers c
  ON c.customer_id = o.customer_id
 AND d.full_date BETWEEN c.eff_start_date
                      AND COALESCE(c.eff_end_date, '9999-12-31');

-- Swap
ALTER TABLE fact_orders RENAME TO fact_orders_old_pit_bug;
ALTER TABLE fact_orders_corrected RENAME TO fact_orders;
```

KEY INSIGHT

IS_CURRENT=TRUE always gives today's version. BETWEEN eff_start AND eff_end gives the version at event time. PIT resol

Scenario 8 [Medium] [CDC & SCD]

SITUATION:

You receive daily full-file CSV extracts from a legacy system. There is NO updated_at column, NO delete flag, and no CDC log available from the source team.

QUESTION:

Design a complete CDC strategy to detect inserts, updates, and deletes from full-file extracts.

APPROACH:

Hash-based CDC + set difference for deletes:

1. Compute MD5(all business columns) at COPY INTO time and store as _src_row_hash
2. Maintain a CDC_STATE control table with last-known hash per business key
3. Compare: NULL hash in state = new record (INSERT). Hash changed = UPDATE. In state but not in today's file = DELETE.

SQL Implementation

```
-- Step 1: Load with row hash
COPY INTO bronze_customers (... , _src_row_hash)
FROM (
    SELECT ...,
        MD5(CONCAT_WS('|', $1,$2,$3,$4,$5,$6,$7)) AS _src_row_hash
    FROM @stage
);

-- Step 2: Detect INSERTs and UPDATES by hash comparison
SELECT src.customer_id,
    CASE
        WHEN ctrl.last_hash IS NULL          THEN 'INSERT'
        WHEN ctrl.last_hash != src._src_row_hash THEN 'UPDATE'
        ELSE                                'NO_CHANGE'
    END AS cdc_operation
FROM bronze_customers src
LEFT JOIN cdc_state ctrl
    ON ctrl.business_key = src.customer_id
   AND ctrl.table_name = 'BRONZE_CUSTOMERS';

-- Step 3: Detect DELETES (in state but NOT in today's file)
SELECT ctrl.business_key AS deleted_id
FROM cdc_state ctrl
WHERE ctrl.table_name = 'BRONZE_CUSTOMERS'
   AND NOT EXISTS (
        SELECT 1 FROM bronze_customers src
        WHERE src.customer_id = ctrl.business_key
              AND src._load_ts >= DATE_TRUNC('day', CURRENT_TIMESTAMP())
    );

-- Step 4: Update CDC state after processing
MERGE INTO cdc_state ctrl
USING (SELECT 'BRONZE_CUSTOMERS' AS table_name,
             customer_id AS bk, _src_row_hash AS h
        FROM bronze_customers WHERE ...) src
ON ctrl.business_key=src.bk AND ctrl.table_name=src.table_name
WHEN MATCHED THEN UPDATE SET ctrl.last_hash=src.h, ctrl.last_updated_at=CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT (table_name,business_key,last_hash) VALUES (src.table_name,src.bk,src.h);
```

KEY INSIGHT

Hash-based CDC = change detection without updated_at. Set difference = delete detection. Works on any full-file extract

PIPELINES

Scenario Questions — Pipelines

Scenario 9 [Hard] [Pipelines]

SITUATION:

Your 5-step task DAG (extract → validate → transform → load → notify) fails at the "transform" step. Analysts are waiting for data.

QUESTION:

What state are the downstream tasks in, what happened to the stream, and how do you recover?

APPROACH:

When transform fails: load and notify are SKIPPED (not FAILED). The stream offset was NOT advanced because the consuming DML never committed. Stream data is preserved. Fix the root cause (stored procedure bug), then EXECUTE TASK on the root or failed task to re-run immediately without waiting for the next scheduled run.

SQL Implementation

```
-- Step 1: Check all task states
SELECT NAME, STATE, ERROR_CODE, ERROR_MESSAGE, SCHEDULED_TIME
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('hour', -2, CURRENT_TIMESTAMP())
))
ORDER BY SCHEDULED_TIME DESC;
-- extract_task:    SUCCEEDED
-- validate_task:   SUCCEEDED
-- transform_task:  FAILED    <-- root cause
-- load_task:       SKIPPED  <-- downstream, not executed
-- notify_task:     SKIPPED  <-- downstream, not executed

-- Step 2: Verify stream is still intact
SELECT SYSTEM$STREAM_HAS_DATA('bronze_db.raw.orders_stream');
-- TRUE: stream data preserved, offset not advanced

-- Step 3: Fix stored procedure (resolve the root cause)
-- ... (debug and fix sp_transform_data)

-- Step 4: Re-run pipeline
-- Option A: Re-run from root (all steps)
EXECUTE TASK extract_task;

-- Option B: Re-run from failed step only (faster)
EXECUTE TASK transform_task;

-- Step 5: Monitor recovery
SELECT NAME, STATE FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('minute', -15, CURRENT_TIMESTAMP())
)) ORDER BY SCHEDULED_TIME DESC;
```

KEY INSIGHT

Failed parent = downstream SKIPPED. Stream preserved (offset unchanged). EXECUTE TASK re-runs from any midpoint without

Scenario 10 [Hard] [Pipelines]**SITUATION:**

You are migrating from Informatica to native Snowflake pipelines. The existing ETL has 15 transformations with complex dependencies. Business requires zero downtime.

QUESTION:

Design a phased zero-downtime migration strategy.

APPROACH:

Parallel run strategy: 1) Map dependencies into a Snowflake task DAG. 2) Build the new pipeline writing to a validation schema (not production). 3) Run both Informatica and Snowflake pipelines in parallel. 4) Compare outputs daily with row counts and checksums. 5) Cut over to Snowflake when validation passes 5 consecutive runs. 6) Monitor for 2 weeks before decommissioning Informatica.

SQL Implementation

```
-- Phase 1: Build Snowflake pipeline in validation schema
CREATE SCHEMA silver_db.validation;

CREATE TASK validate_pipeline WAREHOUSE=transform_wh
  SCHEDULE='USING CRON 0 3 * * * UTC' -- 1 hour after Informatica
AS
  CALL sp_run_full_pipeline('VALIDATION'); -- writes to silver_db.validation

-- Phase 2: Daily comparison
SELECT 'PROD (Informatica)' AS source, COUNT(*), SUM(total_amount),
  SUM(SUM(total_amount)) OVER () AS grand_total
FROM silver_db.facts.fact_orders
WHERE order_date_key >= 20240301
UNION ALL
SELECT 'NEW (Snowflake)', COUNT(*), SUM(total_amount), NULL
FROM silver_db.validation.fact_orders
WHERE order_date_key >= 20240301;

-- Phase 3: Cutover (after 5 consecutive clean validation runs)
-- Suspend Informatica jobs
-- Point Snowflake pipeline to production schema
ALTER TASK validate_pipeline SUSPEND;
CREATE TASK production_pipeline WAREHOUSE=transform_wh
  SCHEDULE='USING CRON 0 2 * * * UTC'
AS CALL sp_run_full_pipeline('PRODUCTION');
ALTER TASK production_pipeline RESUME;

-- Phase 4: Monitor ACCOUNT_USAGE for 2 weeks before decommission
```

KEY INSIGHT

Parallel run + daily validation = risk-free migration. Never cut over without 5+ consecutive clean comparison runs.

SECURITY

Scenario Questions — Security

Scenario 11 [Hard] [Security]

SITUATION:

GDPR erasure request: a customer wants all their data deleted, including 3 years of SCD2 history. Your retention is set to 90 days for all tables.

QUESTION:

Can you truly erase them from Snowflake? What are the constraints?

APPROACH:

True erasure is constrained by Time Travel (up to 90 days) and Fail-safe (7 more days after that). Deleting normally preserves data in Time Travel for 90 days — not truly erased. Solution: set `DATA_RETENTION_TIME=0` on affected tables BEFORE deletion (prevents new Time Travel accumulation). For existing Time Travel data: wait out retention period or contact Snowflake Support. Best long-term solution: tokenize PII at ingestion.

SQL Implementation

```
-- Step 1: Set retention to 0 BEFORE deleting
-- (prevents Time Travel from capturing the new deletes)
ALTER TABLE silver_db.dimensions.dim_customers
  SET DATA_RETENTION_TIME_IN_DAYS = 0;
ALTER TABLE silver_db.facts.fact_orders
  SET DATA_RETENTION_TIME_IN_DAYS = 0;
ALTER TABLE gold_db.customer_mart.customer_360
  SET DATA_RETENTION_TIME_IN_DAYS = 0;

-- Step 2: Delete from ALL tables (all SCD2 versions)
DELETE FROM silver_db.dimensions.dim_customers
  WHERE customer_id = 'C12345';
-- Delete ALL versions (is_current=TRUE and FALSE)

DELETE FROM silver_db.facts.fact_orders
  WHERE sk_customer IN (
    SELECT sk_customer FROM dim_customers_backup
    WHERE customer_id = 'C12345'
  );

-- Step 3: Document for compliance audit
INSERT INTO common_db.audit.gdpr_erasure_log
  (customer_id, request_type, processed_at, processed_by, tables_affected)
VALUES ('C12345', 'ERASURE', CURRENT_TIMESTAMP(), CURRENT_USER(),
  ARRAY_CONSTRUCT('DIM_CUSTOMERS', 'FACT_ORDERS', 'CUSTOMER_360'));

-- Step 4: Restore retention for future data
ALTER TABLE silver_db.dimensions.dim_customers
  SET DATA_RETENTION_TIME_IN_DAYS = 30;
```

KEY INSIGHT

`DATA_RETENTION=0` BEFORE delete prevents new TT capture. Existing TT only clearable by waiting or Snowflake Support. To

Scenario 12 [Hard] [Security]**SITUATION:**

Security audit reveals that data scientists are querying raw Bronze tables directly, bypassing all masking policies and row access policies on Silver/Gold.

QUESTION:

How do you prevent this architecturally while keeping data scientists productive?

APPROACH:

Revoke Bronze and Silver table access from the data scientist role. Grant only Gold layer views (which have masking and row access policies applied). For exploration, create a daily zero-copy clone of production that data scientists can query freely without governance constraints.

SQL Implementation

```
-- Step 1: Revoke direct Bronze/Silver access
REVOKE USAGE ON DATABASE bronze_db FROM ROLE data_scientist_role;
-- If already granted via GRANT ALL:
REVOKE ALL ON ALL TABLES IN DATABASE bronze_db
  FROM ROLE data_scientist_role;
REVOKE ALL ON ALL TABLES IN DATABASE silver_db
  FROM ROLE data_scientist_role;

-- Step 2: Grant only governed Gold layer
GRANT USAGE ON DATABASE gold_db TO ROLE data_scientist_role;
GRANT USAGE ON SCHEMA gold_db.reporting TO ROLE data_scientist_role;
GRANT SELECT ON ALL VIEWS IN SCHEMA gold_db.reporting TO ROLE data_scientist_role;
GRANT SELECT ON ALL TABLES IN SCHEMA gold_db.sales_mart TO ROLE data_scientist_role;

-- Step 3: Daily sandbox for free exploration
CREATE TASK refresh_ds_sandbox
  WAREHOUSE = transform_wh
  SCHEDULE = 'USING CRON 0 6 * * * UTC' -- 6am daily
AS
BEGIN
  DROP DATABASE IF EXISTS ds_sandbox;
  CREATE DATABASE ds_sandbox CLONE gold_db; -- zero-copy clone!
  GRANT ALL ON DATABASE ds_sandbox TO ROLE data_scientist_role;
END;
ALTER TASK refresh_ds_sandbox RESUME;

-- Verify: data scientist cannot see Bronze
USE ROLE data_scientist_role;
SHOW DATABASES; -- bronze_db should not appear
```

KEY INSIGHT

Revoke Bronze/Silver. Governed Gold views = controlled access. Zero-copy sandbox = free exploration without governance

COST

Scenario Questions — Cost

Scenario 13 [Hard] [Cost]**SITUATION:**

Snowflake costs doubled last month. Your account admin left. You have no documentation and no baseline to compare against.

QUESTION:

Walk through your complete investigation using only SQL and Snowflake system views.

APPROACH:

ACCOUNT_USAGE is your forensic trail. 1) METERING_DAILY_HISTORY → find the exact day spend spiked. 2) WAREHOUSE_METERING_HISTORY → find which warehouse drove the spike. 3) QUERY_HISTORY → find the most expensive queries on that warehouse on the spike day. 4) Check AUTO_SUSPEND on all warehouses. 5) Check storage growth.

SQL Implementation

```
-- Step 1: Find the spike day
SELECT USAGE_DATE,
       CREDITS_USED_COMPUTE + CREDITS_USED_CLOUD_SERVICES AS total_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE USAGE_DATE >= DATEADD('month', -3, CURRENT_DATE())
ORDER BY USAGE_DATE;

-- Look for the day costs jumped

-- Step 2: Which warehouse drove the spike?
SELECT WAREHOUSE_NAME, SUM(CREDITS_USED) AS credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= '<spike_date>'
GROUP BY 1 ORDER BY 2 DESC;

-- Step 3: Most expensive queries on spike day
SELECT USER_NAME, TOTAL_ELAPSED_TIME/1000 AS secs,
       PARTITIONS_SCANNED, PARTITIONS_TOTAL,
       LEFT(QUERY_TEXT, 200)
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE WAREHOUSE_NAME = '<culprit_warehouse>'
      AND START_TIME >= '<spike_date>'
ORDER BY TOTAL_ELAPSED_TIME DESC LIMIT 10;

-- Step 4: Check AUTO_SUSPEND on all warehouses RIGHT NOW
SHOW WAREHOUSES;
-- AUTO_SUSPEND = 0 means NEVER suspends = bills 24/7!

-- IMMEDIATE FIX: set auto-suspend on all warehouses
ALTER WAREHOUSE <culprit_wh> SET AUTO_SUSPEND = 60;

-- Step 5: Storage surprises
SELECT TABLE_NAME,
       ROUND(TIME_TRAVEL_BYTES/1e9, 2) AS time_travel_gb,
       ROUND(FAILSAFE_BYTES/1e9, 2) AS failsafe_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.TABLE_STORAGE_METRICS
ORDER BY (TIME_TRAVEL_BYTES+FAILSAFE_BYTES) DESC LIMIT 10;
```

KEY INSIGHT

METERING_DAILY → spike day → WAREHOUSE_METERING → culprit warehouse → QUERY_HISTORY → culprit queries. Most bills: fo

DEBUGGING

Scenario Questions — Debugging

Scenario 14 [Hard] [Debugging]

SITUATION:

A stored procedure runs successfully in dev but fails in prod with "Insufficient privileges to operate on table RAW_ORDERS". Both environments use the same role name.

QUESTION:

Diagnose and fix without granting ACCOUNTADMIN in production.

APPROACH:

EXECUTE AS CALLER means the procedure runs with the calling user's active role. In dev the caller likely has ACCOUNTADMIN or elevated access. In prod, the caller has the ETL service role which doesn't have SELECT on RAW_ORDERS in the prod BRONZE_DB. Fix: either grant the missing privilege to the calling role, or change the procedure to EXECUTE AS OWNER.

SQL Implementation

```
-- Step 1: Check how the procedure is defined
DESC PROCEDURE sp_process_orders();
-- EXECUTE_AS column: CALLER or OWNER

-- Step 2: In prod, check what role actually calls the procedure
SELECT QUERY_TEXT, USER_NAME, ROLE_NAME
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TEXT ILIKE '%sp_process_orders%'
  AND START_TIME > DATEADD('hour', -2, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;

-- Step 3: Check what that role has on the table
SHOW GRANTS TO ROLE etl_svc_role;
-- Does it have SELECT on BRONZE_DB.RAW.RAW_ORDERS? Probably not!

-- Fix Option A: Grant missing privilege to calling role
GRANT SELECT ON TABLE bronze_db.raw.raw_orders
  TO ROLE etl_svc_role;

-- Fix Option B: Change to EXECUTE AS OWNER
-- (runs with procedure creator's privileges, not caller's)
CREATE OR REPLACE PROCEDURE sp_process_orders()
RETURNS STRING LANGUAGE SQL
EXECUTE AS OWNER -- now uses creator's (SYSADMIN) access
AS $$ SELECT COUNT(*) FROM bronze_db.raw.raw_orders; $$;

-- Best practice: ETL procs = EXECUTE AS OWNER
```

KEY INSIGHT

EXECUTE AS CALLER uses calling role's privileges. Dev caller = admin, prod caller = service account. Fix: grant missing

Scenario 15 [Hard] [Debugging]

SITUATION:

Your nightly ETL task has been running for 7 hours. It normally completes in 40 minutes. The warehouse is running. You cannot identify what's blocking it.

QUESTION:

Step-by-step investigation and safe recovery without losing data.

APPROACH:

1) Find the currently running query in INFORMATION_SCHEMA.QUERY_HISTORY. 2) Check for spill in that query (warehouse too small). 3) Check for lock contention in LOCK_WAIT_HISTORY. 4) If query is runaway: SYSTEM\$CANCEL_QUERY (stream offset not advanced = safe to retry). 5) Size up warehouse if spill was the cause, then EXECUTE TASK to re-run.

SQL Implementation

KEY INSIGHT

```
-- Step 1: Find currently running queries
SYSTEM$CANCEL_QUERY kills a specific query. Stream preserved = safe retry. EXECUTE TASK re-triggers immediately after
```

ARCHITECTURE

Scenario Questions — Architecture

Scenario 16 [Hard] [Architecture]

SITUATION:

Your company acquires a competitor who also uses Snowflake on AWS us-east-1 (same as yours). Finance needs consolidated reports within 2 weeks. Data cannot be physically copied.

QUESTION:

Design the fastest, most secure architecture.

APPROACH:

Snowflake Data Sharing is the answer: live data, zero copy, no ETL, set up in hours. Both on same cloud+region = zero egress charges. Provider exposes data via secure views (adds row/column filtering). Consumer creates a database from the share. Build a consolidation UNION view.

SQL Implementation

```
-- Provider (acquired company) side:
CREATE SHARE acquisition_share;
GRANT USAGE ON DATABASE acq_db TO SHARE acquisition_share;
GRANT USAGE ON SCHEMA acq_db.public TO SHARE acquisition_share;

-- Share via SECURE VIEW (not raw tables)
-- Add filtering to expose only agreed-upon data
CREATE SECURE VIEW acq_db.public.shared_orders_v AS
SELECT
    order_id,
    product_category,
    region,
    order_date,
    amount
    -- customer PII NOT included in share
FROM orders;

GRANT SELECT ON VIEW acq_db.public.shared_orders_v
TO SHARE acquisition_share;

ALTER SHARE acquisition_share
ADD ACCOUNTS = your_account_identifier;

-- Consumer (your company) side:
CREATE DATABASE acq_data
FROM SHARE acquired_company.acquisition_share;

-- Build consolidated view
CREATE VIEW gold_db.reporting.v_consolidated_revenue AS
SELECT 'OUR_COMPANY' AS company, *
FROM gold_db.reporting.v_sales_dashboard
UNION ALL
SELECT 'ACQUIRED_CO', *
FROM acq_data.public.shared_orders_v;
```

KEY INSIGHT

Data Sharing = live data, zero copy, zero egress (same region), set up in hours. Consumer pays their own compute. Prov

Scenario 17 [Hard] [Architecture]**SITUATION:**

A healthcare company needs to store PHI (Protected Health Information) in Snowflake. Security requires customer-managed encryption keys so they can revoke access within minutes.

QUESTION:

Which Snowflake edition and features are required? Design the security architecture.

APPROACH:

HIPAA compliance + customer-managed keys (CMK) = Business Critical edition minimum. Tri-Secret Secure: Snowflake uses a composite key (Snowflake's own key + your AWS KMS key). Revoke your KMS key → data is inaccessible. Add network policy, column masking, row access, and ACCESS_HISTORY auditing.

SQL Implementation

```
-- Requires: Business Critical edition

-- 1: Customer-managed key (Tri-Secret Secure)
-- Setup requires Snowflake Support + AWS KMS configuration:
CREATE KEY_MANAGEMENT_INTEGRATION phi_kms
  TYPE          = AWS_KMS
  ENABLED       = TRUE
  KMS_KEY_ID    = 'arn:aws:kms:us-east-1:123:key/your-key-id';
-- Data encrypted with: Snowflake root key + YOUR KMS key
-- Revoke KMS key in AWS → Snowflake cannot decrypt → inaccessible

-- 2: Network isolation (VPC/private link only)
CREATE NETWORK POLICY phi_network
  ALLOWED_IP_LIST = (
    '10.0.1.0/24', -- AWS private link CIDR
    '10.0.2.0/24'  -- Corporate VPN
  );
ALTER ACCOUNT SET NETWORK_POLICY = phi_network;

-- 3: PHI column masking
CREATE MASKING POLICY phi_ssn_mask
AS (v VARCHAR) RETURNS VARCHAR ->
CASE
  WHEN CURRENT_ROLE() IN ('CLINICIAN_ROLE', 'ACCOUNTADMIN') THEN v
  ELSE '***-**- ' || RIGHT(REGEXP_REPLACE(v, '[^0-9]', ''), 4)
END;

ALTER TABLE patients MODIFY COLUMN ssn SET MASKING POLICY phi_ssn_mask;

-- 4: HIPAA audit trail (critical for compliance)
SELECT USER_NAME, QUERY_START_TIME, DIRECT_OBJECTS_ACCESSED
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY
WHERE EXISTS (
  SELECT 1 FROM TABLE(FLATTEN(BASE_OBJECTS_ACCESSED)) f
  WHERE f.value:objectName::VARCHAR ILIKE '%patient%'
)
AND QUERY_START_TIME > DATEADD('day', -90, CURRENT_TIMESTAMP())
ORDER BY QUERY_START_TIME DESC;
```

KEY INSIGHT

HIPAA + CMK = Business Critical required. Tri-Secret Secure = your KMS key + Snowflake key = you control revocation.

Scenario 18 [Hard] [Architecture]

SITUATION:

Tableau dashboards timeout at 30 seconds on complex queries. The analytics warehouse is already at XL. Users are frustrated and productivity is suffering.

QUESTION:

List all possible solutions in order of cost-effectiveness. Which do you implement first?

APPROACH:

Implement in order: 1) Check result cache bypass (CURRENT_DATE in queries = always miss, free fix). 2) Materialized views for most-queried aggregations (moderate cost, high impact). 3) Pre-aggregate into Gold tables on a schedule (transform cost, BI queries free). 4) Separate dedicated BI warehouse (isolation fix). 5) Multi-cluster for concurrency. Bigger warehouse is usually last resort.

SQL Implementation

```
-- 1: Check result cache bypass (FREE fix)
SELECT QUERY_TEXT, COUNT(*) AS runs
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TEXT ILIKE '%CURRENT_DATE%'
  AND WAREHOUSE_NAME = 'ANALYTICS_WH'
  AND START_TIME > DATEADD('day', -7, CURRENT_TIMESTAMP())
GROUP BY 1 ORDER BY 2 DESC;
-- If high count: Tableau refreshes use CURRENT_DATE = always cache miss
-- Fix in Tableau: parameterize the date, use a fixed date filter

-- 2: Materialized view for top dashboard aggregation
CREATE MATERIALIZED VIEW gold_db.reporting.mv_dashboard_kpi AS
SELECT
  d.full_date, d.year, d.month_name,
  c.segment, c.loyalty_tier,
  COUNT(DISTINCT f.order_id) AS orders,
  SUM(f.net_amount) AS revenue,
  AVG(f.net_amount) AS avg_order_value
FROM silver_db.facts.fact_orders f
JOIN silver_db.dimensions.dim_date d ON d.date_key = f.order_date_key
JOIN silver_db.dimensions.dim_customers c ON c.sk_customer = f.sk_customer
GROUP BY 1,2,3,4,5;
-- Tableau queries MV (pre-computed) instead of full fact join
-- Auto-refreshed by Snowflake in background

-- 3: Dedicated BI warehouse (prevents ETL noisy neighbor)
CREATE WAREHOUSE bi_wh
  WAREHOUSE_SIZE='MEDIUM' -- smaller because MV is pre-computed
  MIN_CLUSTER_COUNT=1
  MAX_CLUSTER_COUNT=4 -- multi-cluster for 200 concurrent users
  AUTO_SUSPEND=180 AUTO_RESUME=TRUE;
```

KEY INSIGHT

Fix result cache bypass first (free). Materialized views second (high impact). Multi-cluster for concurrency. Bigger w

QUICK REFERENCE

Most Important Facts for Interview Day

Essential Facts to Know Cold

Topic	Critical Fact
Warehouse billing	Per-second, 60-second MINIMUM per resume. AUTO_SUSPEND=0 = bills 24/7
Result cache	24h, free, no warehouse. Bypassed by CURRENT_TIMESTAMP/RANDOM/UUID_STRING
Local disk cache	Cleared on warehouse suspend/resume. Don't suspend mid-workload
4 required grants	DATABASE usage + SCHEMA usage + TABLE select + WAREHOUSE usage. All 4 required
Stream offset	Advances ONLY on successful DML commit. Failed task = stream preserved = safe retry
Task resume order	RESUME: leaves first, root LAST. SUSPEND: root first
SCD2 FK resolution	JOIN on event_date BETWEEN eff_start AND eff_end. NOT on IS_CURRENT=TRUE
Masking vs row policy	Masking = hides column VALUE. Row access = hides entire ROWS
COPY INTO dedup	64-day ETag history. FORCE=TRUE overrides, risks duplicates
Time Travel vs Fail-safe	TT = you control (SQL). Fail-safe = Snowflake controls (Support only)
Multi-cluster edition	Enterprise edition required. STANDARD policy adds cluster on any queue
Cloud services 10%	Free up to 10% of compute. Above = billed. SHOW commands in loops = main cause
SCD types	0=fixed, 1=overwrite, 2=add row, 3=add column, 4=history table, 6=hybrid 1+2+3
EXECUTE AS OWNER	Proc runs with creator's role. EXECUTE AS CALLER = runs with caller's role
QUALIFY	Snowflake-specific: filter on window functions without subquery
TRANSIENT tables	No Fail-safe (saves 7-day storage). Use for staging tables